

UNIVERZITA PARDUBICE

FAKULTA ELEKTROTECHNIKY A
INFORMATIKY

BAKALÁŘSKÁ PRÁCE

2025

Egor Razboinikov

Univerzita Pardubice
Fakulta Elektrotechniky a Informatiky

Digitalizace evidence pracovní doby
Bakalářská práce

ZADÁNÍ BAKALÁŘSKÉ PRÁCE

(projektu, uměleckého díla, uměleckého výkonu)

Jméno a příjmení:	Egor Razboinikov
Osobní číslo:	I21219
Studijní program:	B0688A140009 Informační technologie
Téma práce:	Digitalizace evidence pracovní doby
Zadávací katedra:	Katedra informačních technologií

Zásady pro vypracování

Dokument evidence pracovní doby vyučujícího vyžaduje provázanost rozvrhu s úvazkem vyučujícího při respektování času přestávek. Cílem bakalářské práce je navrhnout, vytvořit, odzkoušet a předat funkční multiplatformní aplikaci jejíž vstupy jsou: rozvrh vyučujícího, čas na oběd, plán dovolených, státní svátky, školní volna a další. Požadovaným výstupem je PDF dokument (tabulka) se všemi náležitostmi potřebnými pro odevzdání. Grafické rozhraní poskytne pohodlné zadávání, umožní dodatečné ruční modifikace a zobrazování náhledu před vygenerováním PDF dokumentu.

Rozsah pracovní zprávy: **min. 30 stran**
Rozsah grafických prací:
Forma zpracování bakalářské práce: **tištěná**

Seznam doporučené literatury:

PŠENČÍKOVÁ, J. Algoritmizace. Kralice na Hané, Computer Media, 2009.
MAREŠ, Martin a VALLA, Tomáš. Průvodce labyrintem algoritmů 2. vydání. Praha: CZ.NIC, 2022. ISBN 978-80-88168-63-8
PECINOVSKÝ, Rudolf. OOP: naučte se myslet a programovat objektově. Brno: Computer Press, 2010. ISBN 978-80-251-2126-9.

Vedoucí bakalářské práce: **Ing. Radek Matoušek**
Katedra informačních technologií

Datum zadání bakalářské práce: **15. prosince 2024**
Termín odevzdání bakalářské práce: **16. května 2025**

L.S.

prof. Ing. Petr Doležel, Ph.D. v.r.
děkan

Ing. Jan Panuš, Ph.D. v.r.
vedoucí katedry

V Pardubicích dne 28. února 2025

Prohlašuji:

Práci s názvem Digitalizace evidence pracovní doby jsem vypracoval samostatně. Veškeré literární prameny a informace, které jsem v práci využil, jsou uvedeny v seznamu použité literatury.

Byl jsem seznámen s tím, že se na moji práci vztahují práva a povinnosti vyplývající ze zákona č. 121/2000 Sb., o právu autorském, o právech souvisejících s právem autorským a o změně některých zákonů (autorský zákon), ve znění pozdějších předpisů, zejména se skutečností, že Univerzita Pardubice má právo na uzavření licenční smlouvy o užití této práce jako školního díla podle § 60 odst. 1 autorského zákona, a s tím, že pokud dojde k užití této práce mnou nebo bude poskytnuta licence o užití jinému subjektu, je Univerzita Pardubice oprávněna ode mne požadovat přiměřený příspěvek na úhradu nákladů, které na vytvoření díla vynaložila, a to podle okolností až do jejich skutečné výše.

Beru na vědomí, že v souladu s § 47b zákona č. 111/1998 Sb., o vysokých školách a o změně a doplnění dalších zákonů (zákon o vysokých školách), ve znění pozdějších předpisů, a směrnicí Univerzity Pardubice č. 7/2019 Pravidla pro odevzdávání, zveřejňování a formální úpravu závěrečných prací, ve znění pozdějších dodatků, bude práce zveřejněna prostřednictvím Digitální knihovny Univerzity Pardubice.

V Pardubicích dne 9. 5. 2025

Egor Razboinikov v.r.

PODĚKOVÁNÍ

Tady chtěl bych poděkovat svému vedoucímu Ing. Radek Matoušek za hezké vedení bakalářské práce a rychle odpovědi. Také rád bych poděkoval svým rodičům a kamarádům za jejich podporu po celou dobu studia.

ANOTACE

Cílem této bakalářské práce je návrh, implementace a ověření funkčnosti multiplatformní aplikace pro digitalizaci evidence pracovní doby vyučujících. Aplikace umožňuje automatizované zpracování dat, jako jsou rozvrh hodin vyučujícího, stanovené přestávky na oběd, plánované dovolené, státní svátky a školní volna. Výsledným výstupem je detailní PDF dokument, který splňuje formální požadavky fakulty a lze jej snadno upravovat prostřednictvím intuitivního grafického rozhraní. V práci je popsána použitá metodika, výběr vhodných technologií a shrnuty výsledky praktického ověření aplikace.

KLÍČOVÁ SLOVA

C#, .NET, AVALONIA UI, MVVM, AXAML, desktopová aplikace, rozvrh hodin, evidence pracovní doby, generování PDF, automatizace.

TITLE

Digitalization of Working Time Records

ANNOTATION

The aim of this bachelor thesis is to design, implement, and verify a functional cross-platform application for digitizing teachers' working time records. The application automates data processing, including teachers' timetables, designated lunch breaks, vacation schedules, public holidays, and school breaks. The output is a detailed PDF document that meets the faculty's formal submission requirements and can be easily modified via an intuitive graphical interface. The thesis describes the applied methodology, technology selection, and summarizes results of the practical evaluation of the application.

KEYWORDS

C#, .NET, WPF, MVVM, AVALONIA, desktop application, timetable, working time tracking, PDF generation, automation.

OBSAH

SEZNAM ILUSTRACÍ	11
SEZNAM ZKRATEK A ZNAČEK	12
TERMINOLOGIE	14
ÚVOD	15
1 TEORETICKÁ ČÁST	16
1.1 Pravidla evidence pracovní doby	16
1.2 Pravidla a teoretické zásady algoritmu vyrovnávání	17
1.3 Architektura a vzory	19
1.3.1 Co je architektura softwaru	19
1.3.2 MVVM – Model–View–ViewModel	19
1.3.3 MVC – Model–View–Controller	20
1.3.4 Monolit, modulární monolit a mikroslužby	21
1.4 Programovací jazyky	21
1.4.1 Jazyk C#	21
1.4.2 Jazyk SQL	22
1.5 Framework Avalonia UI	24
1.6 Vývojové prostředí	25
1.6.1 Microsoft Visual Studio	25
1.6.2 SQLiteStudio	25
1.7 Knihovny	26
2 PRAKTICKÁ ČÁST	27
2.1 Programovací jazyky	27
2.1.1 Jazyk C#	27
2.1.2 Jazyk XAML	28
2.2 Struktura aplikace	29
2.4 Souborová struktura aplikace	30

2.4.1 Složka Behaviors	31
2.4.2 Složka Controls.....	31
2.4.3 Složka Converters	32
2.4.4 Složka Data	32
2.4.5 Složka Helpers	33
2.4.6 Složka Messages	33
2.4.7 Složka Models.....	34
2.4.8 Složka Services	34
2.4.9 Složka ViewModels	34
2.4.10 Složka Views	34
2.4.11 Složka Kořenové soubory projektu.....	34
2.5 Struktura databáze.....	34
2.5.1 Přehled tabulek	35
2.5.2 Bez cizích klíčů: záměr a důvody	36
2.5.3 Shrnutí rozhodnutí	36
2.6 Uživatelské rozhraní aplikace	37
2.6.1 Hlavní stránka aplikace.....	37
2.6.2 Dialogové okno pro vytvoření události	39
2.6.3 Dialogové okno pro změnu události	40
2.6.4 Vizuální prezentace pro den	41
2.6.5 Vizuální prezentace pro týden	42
2.6.6 Vizuální prezentace pro měsíc	43
2.6.7 Globální nastavení	44
2.6.8 Ukázka PDF	46
2.7 Popsání algoritmů	46
2.7.1 Popis algoritmu BalanceDays	46
2.7.2 Popis algoritmu ze služby pro výpočet pracovní doby	49

ZÁVĚR	51
POUŽITÁ LITERATURA	52

SEZNAM ILUSTRACÍ

Nebyla nalezena položka obsahu.

Obrázek 1: Architektura MVVM.....	20
Obrázek 2 Architektura MVC.....	20
Obrázek 3: Souborová struktura aplikace	31
Obrázek 4 Ukázka kódu třídy AppDbContext.....	33
Obrázek 5 Levý panel aplikace.....	38
Obrázek 6 Pravý panel aplikace.....	39
Obrázek 7 Dialogové okno pro vytvoření.....	40
Obrázek 8 Dialogové okno pro změnu	41
Obrázek 9 Vizuální prezentace pro den	42
Obrázek 10 Vizuální prezentace pro týden.....	43
Obrázek 11 Vizuální prezentace pro měsíc	44
Obrázek 12 Globální nastavení.....	45
Obrázek 13 Ukázka PDF	46

SEZNAM ZKRATEK A ZNAČEK

EPD – evidence pracovní doby

FPD – fond pracovní doby

D – dovolená

PC – pracovní cesta

N – nemoc

OČR – ošetřování člena rodiny

L – lékař

S – svátek

CSV – Comma-Separated Values

PDF – Portable Document Format

GUI – Graphical User Interface

STAG – Studijní agenda

MVC – Model – View – Controller

OS – operační systém

DB – Databáze

DI – Dependency Injection

IoC – Inversion of Control

EF Core – Entity Framework Core

FK – Foreign Key

UI – User Interface

JIT – Just-In-Time

MVVM – ModelView – View – Model

ORM – Object–Relational Mapping

PDF – Portable Document Format

UI – User Interface

CLR – Common Language Runtime

XAML – eXtensible Application Markup Language

AXAML – Avalonia eXtensible Application Markup Language

RI – Run-Time Information

CI/CD – Continuous Integration / Continuous Deployment

TERMINOLOGIE

Avalonia UI: Multiplatformní framework pro vývoj uživatelského rozhraní aplikací, který umožňuje tvorbu vizuálně konzistentních aplikací běžících na různých operačních systémech, jako jsou Windows, macOS a Linux, za pomoci XAML a programovacího jazyka C#.

Framework: Sada nástrojů, knihoven a standardizovaných postupů, která usnadňuje proces vývoje softwaru poskytováním struktury a základních funkcionalit, umožňujících rychlou implementaci úloh bez nutnosti opakování vytváření stejného kódu.

Relační databáze: Typ databáze ukládající informace v tabulkách s jasně definovanými vztahy (např. vazby jedna k jedné, jedna k mnoha), umožňující efektivní ukládání, vyhledávání a správu dat.

Programovací paradigma (MVVM): Model-View-ViewModel představuje návrhový vzor programování, který jasně odděluje uživatelské rozhraní (View), datovou logiku (Model) a komunikační logiku (ViewModel), což vede k lepší udržitelnosti a škálovatelnosti aplikace.

Uživatelské rozhraní (UI): Vizuální a interaktivní část aplikace, navržená pomocí frameworku Avalonia UI, která umožňuje uživatelům jednoduše komunikovat s programem prostřednictvím grafických prvků, jako jsou tlačítka, formuláře, menu a další ovládací prvky.

Integrované vývojové prostředí (IDE): Softwarový nástroj kombinující editor kódu, kompilátor, debugger a nástroje pro správu verzí, používaný při vývoji aplikací.

Systémy pro správu verzí (VCS): Nástroje umožňující sledování změn kódu, uchování historie verzí, spolupráci mezi vývojáři a obnovu předchozích verzí kódu v případě potřeby.

ÚVOD

Evidence pracovní doby je formální dokument, který vyučující odevzdává pravidelně za jednotlivé měsíce. Správné rozložení hodin podle rozvrhu, přestávek a dalších pravidel není vždy jednoduché. Proto je vhodné celý proces digitalizovat a co nejvíce automatizovat.

Cílem této bakalářské práce je navrhnout, vytvořit, otestovat a předat funkční multiplatformní aplikaci, která vyučujícímu pomůže připravit EPD. Vstupem je pouze rozvrh vyučujícího exportovaný ze systému STAG ve formátu CSV. Aplikace z těchto dat sestaví přehled pracovní doby a vygeneruje výstupní dokument ve formátu PDF připravený k odevzdání. Součástí řešení je jednoduché grafické rozhraní, které umožní pohodlné zadávání, náhled a ruční doplnění či úpravu informací, které v rozvrhu chybí, například dovolená, služební cesta nebo nemoc. Program je navržen tak, aby byl použitelný na systémech Windows i Linux bez virtualizace.

Navržený algoritmus automaticky převádí data z rozvrhu vyučujícího (CSV) do podoby, která odpovídá požadavkům na evidence pracovní doby (EPD – evidence pracovní doby). V povolených mezích umí upravit časy příchodu, odchodu a polední přestávky, hlídá kolize s výukou a vyrovnává nerovnoměrnosti mezi dny a týdny. Po zpracování jsou data v konzistentním a formálně správném stavu a lze z nich přímo vygenerovat tabulku připravenou k exportu do formátu PDF. Podrobná pravidla i přesná struktura výstupní tabulky jsou popsány v samostatné kapitole.

Motivací práce je zjednodušit administrativu vyučujícího a ušetřit čas. Automatické zpracování z rozvrhu a přehledné grafické rozhraní mají vyučujícímu umožnit rychlou kontrolu a drobné doladění detailů. Přínosem je také jednotný a formálně správný výstup v PDF.

Teoretická část popisuje použitý model architektury, klíčové algoritmy pro rozkládání pracovní doby a pravidla, která aplikace hlídá. Praktická část se zaměřuje na analýzu požadavků, návrh uživatelského rozhraní, strukturu dat, import rozvrhu ze systému STAG, generování PDF a testování na reálných scénářích. V závěru jsou zhodnoceny dosažené cíle a návrhy dalšího rozvoje.

1 TEORETICKÁ ČÁST

1.1 Pravidla evidence pracovní doby

Tato podkapitola sjednocuje pravidla, podle nichž má výsledný výkaz odpovídat internímu vzoru EPD. Formulace jsou záměrně jednoduché a bez vzorců.

Týdenní součet hodin. Každý pracovní týden má v součtu odpovídat standardní týdenní pracovní době. U plného úvazku je to čtyřicet hodin; u zkrácených úvazků se použije hodnota daná pracovní smlouvou.

Týden se svátkem. Je-li v týdnu státní svátek nebo jiný den pracovního klidu, zkracuje se požadovaný týdenní součet o osm hodin za každý takový den. Jeden svátek v týdnu tedy znamená celkový součet třicet dva hodin.

Neúplné týdny a týdny rozdělené mezi měsíce. Pokud má týden méně než pět pracovních dnů, například na začátku či na konci měsíce, celkový součet hodin za tento týden odpovídá počtu pracovních dnů v týdnu; každý takový den se hodnotí jako osm hodin.

Denní záznam „Odpracováno“ a přesčas. Hodiny nad rámec běžného dne se zapisují do sloupce *Odpracováno*. Sloupec *Přesčas* se nevyplňuje; přesčasem se rozumí práce nad týdenní limit. Napracování volna se za přesčas nepovažuje. Součet *Odpracováno* za týden musí vždy odpovídat požadované týdenní hodnotě podle pravidel výše.

Začátek, přestávky a konec pracovní doby. Začátek pracovní doby, přestávka a konec pracovní doby musí být v souladu s výukou uvedenou v systému STAG. Pokud na sebe výuka bezprostředně navazuje, přestávka se čerpá na jejich rozhraní. V docházce se obvykle uvádí osm hodin práce a půlhodinová přestávka.

Služební cesta. V pracovním dni se pracovní cesta vykazuje jako běžná osmihodinová přítomnost. Cestovní náhrady se do EPD neuvádějí, řeší se v cestovním příkazu. Práce během pracovní cesty o víkendu se do pracovní doby v EPD nezapisuje.

Neodpracované hodiny a poznámky. Do sloupce *Neodpracováno* se uvádí součet neodpracovaných hodin z důvodu dovolené nemoci, ošetřování člena rodiny nebo návštěvy lékaře. Státní svátky jsou ve výkazu již označeny.

Měsíční kontrola vůči fondu. Součet hodin za měsíc musí odpovídat měsíčnímu fondu pracovní doby. Kontrola vychází z toho, že odpracované hodiny spolu s případnými položkami v *Přesčas*

(tyto se v praxi nevyužívají) a s neodpracovanými hodinami dávají dohromady hodnotu FPD. Ve vzorové tabulce tuto kontrolu zajišťují souhrnné buňky v řádku „Celkem“.

1.2 Pravidla a teoretické zásady algoritmu vyrovnávání

Tato podkapitola popisuje teoretický rámec, podle kterého se algoritmus chová. Nejde o návod krok za krokem, ale o pravidla, priority a omezení, která zajišťují, že výstup je vhodný pro EPD.

- **Cíl a kritéria úspěchu.**

Cílem je převést data z rozvrhu vyučujícího do podoby, která splní týdenní i měsíční požadavky, bude bez vnitřních rozporů a zároveň bude respektovat povolené překryvy. Druhým cílem je co nejméně se odchýlit od skutečného rozvrhu.

- **Priorita stavů.**

Algoritmus uvažuje pevné pořadí významu stavů dne. Nejvyšší prioritu mají svátky a jiné dny pracovního klidu. Následují stavy nepřítomnosti, tedy dovolená, nemoc, ošetřování člena rodiny a návštěva lékaře. Poté následuje pracovní cesta, a nakonec běžná práce podle rozvrhu. Stav s vyšší prioritou přepisuje stav s nižší prioritou. Z toho plyne, že nepřítomnost může překrýt výuku a do EPD se zapíše právě nepřítomnost. Pracovní cesta se zapisuje jako běžná osmihodinová přítomnost bez ohledu na rozvrh.

- **Tvrdá pravidla.**

Pokud se ve stejném čase potká práce podle rozvrhu se zvláštním stavem dne, přednost má zvláštní stav. Do EPD se v takovém úseku zapíše pouze tento stav; překrývající se práce se do Odpracováno nezapočítá. U pracovních cest se eviduje standardní osmihodinová přítomnost bez ohledu na rozvrh.

Jestliže se v jednom dni, ale v různých časech, vyskytne jak práce, tak zvláštní stav, výkaz má obsahovat oba oddělené bloky: odpracovaný čas se uvede do sloupce Odpracováno a neodpracovaný čas do sloupce Neodpracováno s příslušnou značkou zvláštního stavu. Tyto bloky se nesmí překrývat, musí tvořit smysluplné a souvislé úseky dne a po zaokrouhlení na sjednocenou časovou mřížku musí zůstat vnitřně konzistentní.

Současně platí všechny požadavky z části 1.1: týdenní součty, zacházení s neúplnými týdny a týdny se svátky a měsíční fond pracovní doby. Kombinace práce a částečné nepřítomnosti v jednom dni je tedy přípustná, pokud jsou splněny časové podmínky a zůstává zachována správnost na úrovni týdne i měsíce.

- **Měkká pravidla.**

Preferuje se minimální zásah do rozvrhu a rovnoměrné, malé změny. Přednost má úprava délky a umístění polední přestávky před posunem příchodu nebo odchodu. Pokud existuje více řešení se stejnou kvalitou, volí se to, které znamená méně odchylek od rozvrhu.

- **Povolené zásahy.**

Algoritmus smí jemně posouvat časy příchodu a odchodu v mezích, které nastaví uživatel, a může upravovat délku a umístění polední přestávky. Stavy nepřítomnosti a pracovní cesta jsou považovány za pevné body dané uživatelem; algoritmus je sám nezkracuje ani nerozděluje, pouze s nimi počítá při vyrovnávání.

- **Priority rozhodování.**

Nejprve se musí splnit tvrdá pravidla. Následně se zachovávají časy výuky, pokud nejsou překryty stavem s vyšší prioritou, typicky nepřítomností. Teprve poté se upravují přestávky a až nakonec okraje dne. Pokud je to nezbytné pro dosažení měsíčního fondu pracovní doby, mohou se vyrovnávat hodiny mezi neúplnými týdny na začátku a na konci měsíce; plné týdny se mění až jako poslední.

- **Stabilita a determinismus.**

Shodné vstupy musejí dávat shodný výstup. Malá změna na vstupu má vyvolat pouze malou změnu na výstupu. Díky tomu je výsledek předvídatelný a snadno kontrolovatelný.

- **Korektnost vůči měsíčnímu fondu.**

Součet hodin za měsíc musí odpovídat měsíčnímu fondu pracovní doby. Nestačí-li úpravy uvnitř jednotlivých týdnů, je přípustné provést vyvážené změny mezi neúplnými týdny na okraji měsíce. Zásahy do plných týdnů přicházejí na řadu až v krajním případě.

- **Transparentnost a dohledatelnost.**

Každou automatickou úpravu má být možné jednoduše vysvětlit, například: „krátká nepřítomnost překryla výuku; chybějící čas byl dorovnán v jiném dni téhož týdne“. Tato zásada podporuje důvěru ve výsledek a usnadňuje ruční doladění v grafickém rozhraní.

- **Předpoklady a omezení.**

Algoritmus vychází z rozvrhu exportovaného ze Studijní agenty ve formátu CSV. Nepřítomnosti a pracovní cesty zadává uživatel; algoritmus je automaticky negeneruje.

- **Ukončení a bezpečné selhání.**

Pokud nelze splnit všechna pravidla současně, například při velmi hustém rozvrhu nebo při větším počtu pevných absencí v jednom týdnu, algoritmus zůstane v nejbližším dosažitelném konzistentním stavu, problém zřetelně označí a vyžádá ruční zásah. Nikdy nevytváří hodiny, které by porušily tvrdá pravidla.

1.3 Architektura a vzory

Tato práce řeší desktopovou multiplatformní aplikaci s lokálním **GUI**, importem **CSV** a exportem do **PDF**. Nevyžaduje síťovou kompozici služeb ani samostatné škálování částí systému. Z těchto důvodů volím **modulární monolit** (jasně oddělené moduly: import CSV, algoritmus vyrovnávání, generátor PDF, GUI) nasazovaný jako jeden celek. Přístup minimalizuje provozní režii, zjednodušuje testování a zrychluje vývoj [17], [19]. Na úrovni prezentace je zvolen vzor **MVVM**, protože přináší lepší testovatelnost a čistotu návrhu díky datovým vazbám a příkazům [3], [4].

1.3.1 Co je architektura softwaru

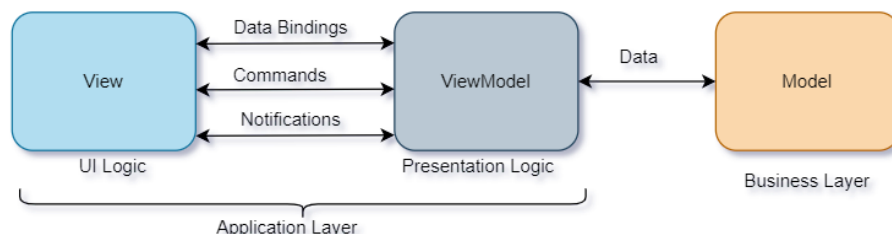
V této práci chápu **architekturu softwaru** jako sadu klíčových rozhodnutí o struktuře systému: jaké má komponenty, jak spolu komunikují, jak jsou rozdělené odpovědnosti a jak budou dosaženy systémové vlastnosti (např. modifikovatelnost, spolehlivost, bezpečnost). Toto pojetí odpovídá definicím uváděným Software Engineering Institute a běžné praxi v softwarovém inženýrství [13].

1.3.2 MVVM – Model–View–ViewModel

MVVM je architektonický vzor pro aplikace s XAML/GUI, který odděluje zobrazení (View) od prezentační logiky (ViewModel) a doménových dat/služeb (Model). Klíčovou roli hrají data binding a commands, takže změny stavu ve ViewModelu se automaticky promítají do UI a naopak. Výsledkem je testovatelný a málo provázaný návrh [3], [4].

Jak MVVM používám v této práci. Obrazovky (AXAML v Avalonia UI) obsahují pouze prezentační vrstvu; logika je v testovatelných třídách ViewModel. Pro reaktivní stav a příkazy využívám ReactiveUI, které cílí na MVVM a podporuje deklarativní práci se stavem [4], [12].

Díky tomu lze prezentační logiku testovat bez běžícího UI a současně zachovat čisté oddělení Modelu, View a ViewModelu [3], [4].

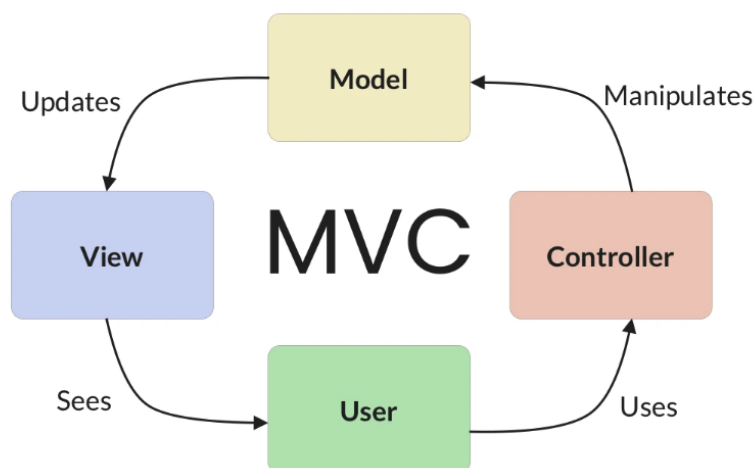


Obrázek 1: Architektura MVVM

1.3.3 MVC – Model–View–Controller

Existuje ještě jedna populární architektura MVC, uvedu její definici, abychom pochopili rozdíly a důvod, proč jsem použil MVVM.

MVC rozděluje aplikaci na model, pohled a kontroler. Kontroler zpracovává vstup uživatele a rozhodne, který pohled vykreslit. V prostředí webu (např. ASP.NET MVC) jde o ověřený přístup, který přináší jasné oddělení zodpovědností ve vrstvě prezentace [11]. Oproti tomu u desktopových XAML aplikací (Avalonia, WPF, MAUI) dává větší smysl MVVM, protože nativně využívá data binding a příkazy – tedy mechanismy, které v MVC typicky řeší kontroler ručně [3], [11].



Obrázek 2 Architektura MVC

1.3.4 Monolit, modulární monolit a mikroslužby

Monolitická aplikace je nasazována jako jediná jednotka (jeden proces/binárka). Interně může být členěná do vrstev a knihoven, navenek ale tvoří jeden nasaditelný celek. Oficiální materiály .NET ji uvádějí jako běžnou a srozumitelnou volbu pro menší až středně velké systémy [17].

Mikroslužby systém rozkládají na sadu samostatně nasaditelných služeb s nezávislými cykly vývoje a škálováním. Přinášejí však vyšší komplexitu (komunikace, konzistence, nasazování) a vyžadují odlišné provozní postupy [18], [20].

Pro nový projekt je často racionální strategie „**monolith-first**“: nejprve vybudovat dobře strukturovaný monolit a teprve při prokázané potřebě přejít na mikroslužby [19]. Tím se vyhneme předčasné distribuované složitosti.

1.4 Programovací jazyky

Tato kapitola vysvětluje, proč je jádrem řešení jazyk **C#** na platformě **.NET**, jaké výhody to přináší pro multiplatformní desktopovou aplikaci s **GUI**, a jaké existují rozumné alternativy. Cílem je ukázat, že volba jazyka podporuje požadavky práce: přenositelnost mezi Windows a Linux, dobré knihovny pro MVVM, práci s CSV/SQLite a spolehlivou tvorbu PDF.

1.4.1 Jazyk C#

C# je moderní, obecně použitelný jazyk s důrazem na produktivitu i výkon. Je součástí ekosystému **.NET** a má širokou podporu nástrojů i knihoven. Oficiální dokumentace popisuje **C#** jako multiplatformní jazyk používaný napříč **.NET** workloady (desktop, web, cloud) a zaměřený na čitelný a bezpečný kód [1]. Platforma **.NET** je zdarma, otevřená a běží na Windows, Linuxu i macOS, takže podporuje multiplatformní vývoj jedním kódem [2].

Pro uživatelské rozhraní používám Avalonia UI, které přirozeně staví na vzoru **MVVM** (Model–View–ViewModel) a na datových vazbách. Oficiální dokumentace Avalonie popisuje **MVVM** a oddělení View a ViewModelu přes binding a DI, což vede k čistému návrhu a snadnému testování prezentační logiky [3]. Reaktivní přístup zajišťuje **ReactiveUI**, které je přímo zaměřeno na **MVVM** a testovatelný, reaktivní kód [4].

Pro lokální úložiště je vhodná **SQLite**, která používá standardní SQL a dobře se integruje do **.NET**. Oficiální dokumentace přesně popisuje, jaké části SQL **SQLite** podporuje, a poskytuje kompletní syntaxi i funkce [5]. To umožňuje jednoduchou práci s tabulkami, součty a filtry, které aplikace potřebuje pro výkazy.

Pro export používám knihovnu **QuestPDF**, která nabízí „fluent“ API v C# a dobře se hodí pro deterministické rozvržení tabulek a textu. Domovská stránka a dokumentace uvádí, že knihovna je určena právě pro generování PDF v .NET aplikacích [6].

Alternativou je **TypeScript/JavaScript** s frameworkem **Electron**: umožňuje psát desktop v JS/HTML/CSS a běží napříč platformami díky Chromiu a Node.js. Je to rozšířená cesta, ale přináší vyšší nároky na paměť a jiný model UI než XAML/MVVM [7]. Další možnost je **Python + Qt (PySide6/Qt for Python)** – oficiální vazby na Qt umožňují psát multiplatformní GUI v Pythonu; tato cesta je silná v rychlém prototypování, ale pro můj projekt oceňuji silnou typovou kontrolu C# a hladkou integraci s .NET ekosystémem [8], [9]. Z „JVM světa“ je zajímavá **Kotlin Compose Multiplatform**, která sdílí deklarativní UI napříč desktopem, Androidem, iOS a webem; pro tuto práci ale nepotřebuji mobilní cíle a dávám přednost stacku C#/.NET kvůli už zvoleným knihovnám a zkušenostem [10].

Shrnutí volby. C#/.NET dává v kombinaci s Avalonia UI a ReactiveUI ucelený, testovatelný a multiplatformní základ. SQLite pokrývá lokální databázi se srozumitelným SQL a QuestPDF řeší spolehlivou tvorbu tiskových výstupů. Tyto technologie společně s MVVM zjednodušují vývoj, testování i údržbu a přesně cílí na potřeby této práce [1] – [6].

1.4.2 Jazyk SQL

Tato podkapitola shrnuje teorii potřebnou pro práci s **SQL** (Structured Query Language – strukturovaný dotazovací jazyk) v kontextu lokální databáze **SQLite** a její integrace do aplikace v .NET.

Data jsou uložena v tabulkách (sloupce/řádky). Každá tabulka má mít **PK** (Primary Key – primární klíč) pro jednoznačnou identifikaci řádku. Vztahy mezi tabulkami se vyjadřují přes **FK** (Foreign Key – cizí klíč). SQLite cizí klíče podporuje a jejich kontrolu lze zapnout, aby se udržela referenční integrita [5].

- **DDL. Data Definition Language** – definiční část jazyka slouží k vytváření a změnám tabulek a schématu (CREATE TABLE, ALTER TABLE).
- **DML. Data Manipulation Language** – manipulační část jazyka pracuje s obsahem tabulek (INSERT, UPDATE, DELETE).
- **DQL. Data Query Language** – dotazovací část jazyka vrací data (SELECT, WHERE, ORDER BY, GROUP BY).

Omezení a integrita dat. Kvalitu dat pomáhají držet **NOT NULL**, **UNIQUE**, **CHECK** a **FK**. Tato omezení zabraňují záznamům bez povinných údajů, duplicitám či nelogickým hodnotám (např. záporný počet hodin). Význam omezení a jejich syntaxi SQLite výslovně definuje [5].

Transakce a ACID. Změny se provádějí v transakcích (**BEGIN**, **COMMIT**, **ROLLBACK**). Cílem je splnit vlastnosti **ACID** (Atomicity – atomičnost, Consistency – konzistence, Isolation – izolace, Durability – trvalost), aby například částečně provedené úpravy nepoškodily data. SQLite popisuje chování transakcí a jejich režimy (*deferred/immediate/exclusive*) ve své dokumentaci [5].

Dotazování a výkon. Pro přehledy a součty (měsíční fondy, týdenní součty) jsou užitečné agregace (**SUM**, **COUNT**, **AVG**), seskupení (**GROUP BY**) a filtry (**WHERE**). SQLite podporuje **CTE** (Common Table Expressions – dočasné pojmenované výsledky), včetně rekurzivních dotazů, a **window functions** (okenní funkce) pro pokročilé agregace [5]. Výkon výrazně ovlivňují **indexy** (např. na datu a klíčích), které zrychlují hledání a spojování tabulek [5].

Datové typy v SQLite (typová afinita). SQLite používá tzv. *manifest typing*: interně rozlišuje **storage classes** (**INTEGER**, **REAL**, **TEXT**, **BLOB**, **NULL**) a sloupce mají **type affinity** (např. „**INTEGER**“, „**TEXT**“). To je flexibilní, ale vyžaduje pečlivé ošetření formátů (např. datum/čas) a konzistentní validace v aplikaci [5].

Datum a čas, lokalizace a porovnávání řetězců. Datумы a časy se v SQLite ukládají obvykle jako ISO-8601 text, číslo (**UNIX epoch**) nebo juliánské číslo. Pro české prostředí je důležité i **collation** (způsob porovnávání řetězců). SQLite k těmto oblastem uvádí doporučené postupy a vestavěné funkce (*date*, *strftime* aj.) [5].

Bezpečnost dotazů. Při práci s uživatelskými vstupy je nutné používat **parametrizované dotazy** (nebo **ORM**) a vyhnout se skládání SQL z řetězců, aby nevznikl prostor pro **SQL injection**. V .NET to řeší připravené parametry nebo **ORM** vrstva.

Integrace do .NET. V .NET lze pracovat přímo přes **ADO.NET** nebo přes **ORM Entity Framework Core** (**EF Core**), pro které existuje oficiální **SQLite provider** [21].

SQL/SQLite poskytuje stabilní a přenositelné úložiště pro výkazy a přehledy. Díky omezením a transakcím drží integritu dat, díky indexům a agregacím umožňuje rychlé výpočty součtů. V kombinaci s .NET (a případně **EF Core**) je práce s daty jednoduchá a bezpečná [5], [21].

1.5 Framework Avalonia UI

Tato kapitola shrnuje teoretické základy **Avalonia UI** – otevřeného frameworku pro tvorbu uživatelských rozhraní v ekosystému **.NET**. Zaměřuji se na koncepty, které jsou důležité pro návrh, implementaci a testování aplikace s architekturou **MVVM – Model–View–ViewModel**.

Avalonia je XAML-like framework pro .NET, který umožňuje tvořit deklarativní uživatelská rozhraní, oddělit je od logiky a přirozeně používat MVVM. Dokumentace výslovně pracuje s MVVM jako doporučeným vzorem a popisuje vazby mezi View a ViewModel pomocí **data bindingu** a příkazů [3].

Uživatelské rozhraní je definováno v souborech **AXAML** (varianta XAML používaná Avalonií). Tyto „View“ neobsahují aplikační logiku; ta je soustředěna do tříd **ViewModel**, které vystavují vlastnosti a příkazy. Doménová data a služby tvoří **Model**. Tato separace je jádrem MVVM a vede k nízké provázanosti a lepší testovatelnosti [3].

Vazby (binding) propojují vlastnosti ViewModelů s prvky UI. Příkazy (command) zajišťují spouštění akcí z UI bez nutnosti psát kód-behind. Tyto mechanismy snižují množství lepidlového kódu a udržují prezentaci deklarativní [3].

Pro práci se stavem, validacemi a událostmi využívám knihovnu **ReactiveUI**. Poskytuje reaktivní vlastnosti, příkazy a pozorovatelné toky dat, které se přirozeně setkávají s MVVM (ViewModel je snadno testovatelný bez běžícího UI) [4], [12].

Koncept **Behaviors** je známý z WPF a slouží k přidání opakovatelných interakcí k prvkům UI bez psaní kódu do View. Microsoft jej popisuje pro WPF [14]. V Avalonia UI je k dispozici knihovna **Avalonia.Xaml.Behaviors** [15] a dokumentace s ukázkami použití (např. spouštění příkazů na určité události, podmíněné chování apod.) [16]. Behaviors pomáhají udržet View čisté a opakovaně použitelné.

Stejně jako jiné XAML frameworky umožňuje Avalonia definovat **styly** a **kontrolní šablony** (templates) a sdílet **zdroje** (resources). Díky tomu lze UI přizpůsobit a udržovat konzistentní vzhled bez zásahu do logiky. V praxi to zapadá do MVVM, protože vzhled se mění ve View, zatímco chování zůstává ve ViewModelu [3].

MVVM v Avalonia UI přirozeně podporuje **unit testy** ViewModelů, jelikož prezentační logika je oddělena od vykreslování. Změny ve vzhledu (AXAML) lze provádět bez zásahu do logiky a naopak, což zrychluje iteraci a snižuje riziko regresí [3], [4].

Avalonia UI přináší deklarativní tvorbu GUI, čisté oddělení vzhledu a logiky (MVVM), možnost reaktivní implementace s ReactiveUI a rozšiřitelnost přes Behaviors. Tyto vlastnosti jsou pro můj projekt klíčové: usnadňují návrh náhledu EPD tabulek, validací a stavové logiky, a současně drží kód přehledný a testovatelný [3], [4], [14] – [16].

1.6 Vývojové prostředí

V této kapitole popisuji nástroje, ve kterých probíhal vývoj aplikace, a důvody jejich volby. Cílem je zdokumentovat prostředí tak, aby bylo možné práci snadno zopakovat na jiném stroji i v budoucnu. Používám **IDE – Integrated Development Environment (integrováné vývojové prostředí)** pro vývoj v **C#/.NET** a samostatný nástroj pro správu lokální databáze **SQLite**.

1.6.1 Microsoft Visual Studio

Microsoft Visual Studio patří mezi přední integrovaná vývojová prostředí (IDE), která umožňují vývojářům pokrýt celý životní cyklus softwaru – od psaní a ladění kódu až po nasazení a správu verzí. Díky podpoře řady programovacích jazyků (C#, C++, Visual Basic, F#, Python a dalších) lze v jednom řešení snadno kombinovat různé technologie. Visual Studio poskytuje funkce jako IntelliSense pro inteligentní dokončování kódu, refaktoring pro rychlou úpravu struktury projektů, integrovaný debugger s možností krokování napříč vlákny a profilování výkonu aplikace.

IDE je navrženo tak, aby plynule spolupracovalo s distribuovanými systémy správy verzí – od Git a GitHub přes Azure DevOps až po Subversion nebo Mercurial. Umožňuje vizuální sledování změn, práci s větvemi a slučování kódu přímo v prostředí, bez nutnosti samostatných nástrojů. Rozšíření a doplňky navíc umožňují připojit nástroje pro Docker, Kubernetes, testování jednotek, analýzu kvality kódu či integraci s cloudovými službami Azure, AWS nebo Google Cloud.

Visual Studio dále zahrnuje návrháře UI pro WPF, WinForms i webové technologie, integrované editory pro SQL, JSON či XML a správce balíčků NuGet pro snadné přidávání třetích stran. Funkce Live Share navíc umožňuje vzdálenou kolaboraci v reálném čase, což vede k efektivnější týmové práci a rychlejšímu řešení problémů.

1.6.2 SQLiteStudio

SQLiteStudio je multiplatformní grafické rozhraní určené ke správě SQLite databází, které nevyžaduje instalaci serveru a pracuje přímo se souborem databáze. Jeho přehledné GUI

dovoluje rychle vytvářet, měnit či mazat tabulky, indexy a pohledy, prohlížet i upravovat záznamy a zároveň exportovat výsledky dotazů do formátů CSV, SQL či HTML.

Vestavěný SQL editor podporuje syntaktické zvýraznění, automatické doplňování a historii dotazů, což usnadňuje psaní i ladění složitých SQL příkazů. SQLiteStudio umí zobrazit datový model v podobě ER diagramu, spravovat trigger funkce a definovat cizí klíče. Součástí nástroje je také vizuální návrhář schématu a debugger pro procedurální rozšíření SQLite.

Pro zálohování a obnovu databáze stačí jediný klik, nástroj navíc umožňuje hromadnou úpravu dat, filtrování či seskupování přímo v tabulce. Díky podpoře importu dat z externích zdrojů (CSV, SQL dump) a integrovanému reportingu je SQLiteStudio oblíbenou volbou pro rychlý vývoj prototypů, správu lokálních databází i pro nasazení ve vestavěných systémech.

1.7 Knihovny

Knihovny představují znovupoužitelné softwarové komponenty, které rozšiřují funkcionalitu aplikací bez nutnosti implementace všech částí kódu od začátku. Mohou obsahovat hotové funkce, třídy či šablony pro řešení specifických úloh.

V prostředí .NET se pro správu knihoven využívá systém NuGet, jenž umožňuje snadnou instalaci, aktualizaci a odstraňování závislostí. Použití knihoven zrychluje vývoj, snižuje chybovost a zajišťuje vyšší kvalitu softwaru, protože vývojáři pracují s prověřenými a otestovanými komponentami.

2 PRAKTICKÁ ČÁST

V této kapitole je podrobně popsána realizace aplikace, která umožňuje evidovat a vyhodnocovat pracovní dobu vyučujících. Zaměřuje se na volbu programovacích jazyků, použitý framework, architekturu aplikace a návrhový vzor MVVM. Dále jsou rozebrány klíčové části implementace včetně práce s databází, importu rozvrhu, generování výstupů a uživatelského rozhraní.

2.1 Programovací jazyky

V projektu používám kombinaci C# (aplikační logika v prostředí .NET) a AXAML. C# obstarává práci s daty, algoritmy a integrace (import CSV, přístup do SQLite, generování PDF), zatímco AXAML deklarativně popisuje obrazovky a jejich vazby na MVVM. Tato dvojice udržuje čisté oddělení prezentační a logické vrstvy a zároveň umožňuje těsnou integraci přes data binding a příkazy.

2.1.1 Jazyk C#

C# tvoří páteř řešení. V tomto jazyce jsou implementované:

Modely mapované přes **Entity Framework Core** na **SQLite**. Schéma odpovídá tomu, co je v databázi:

- **Events** – jednotná tabulka pro všechny denní události (výuka, práce, svátky a jiné dny pracovního klidu, dovolená, nemoc, ošetřování člena rodiny, lékař, služební cesta).
Důležité sloupce: *StartTime*, *EndTime*, *EventType*, *AllDay*, *ArrivalTime*, *DepartureTime*, *LunchStart*, *LunchEnd*, *IsAutoGenerated*, *ParentEventId*, *HasCollision*, *ImportBatchId*, *ImportLabel*.
- **GlobalSettings** – výchozí pravidla a limity (roční/semestrální hodnoty, defaultní příchody/odchody a obědy po dnech, minimální a maximální délka přestávky, názvy auto-událostí před/po obědě).
- **UserSettings** – uživatelská přenastavení pro konkrétní datum (příchod, odchod, začátek a konec oběda).
- **__EFMigrationsHistory** – historie migrací schématu.

Služby:

- import rozvrhu ze souboru CSV (načtení šarže, typování událostí, uložení do Events),

- vyrovnávání a validace hodin (pravidla z EPD – evidence pracovní doby; generování podpůrných bloků přestávky, jemné posuny v povolených mezích; označení kolizí),
- export do PDF přes QuestPDF (měsíční tabulka s hlavičkami, součty, zvýraznění svátků a celodenních stavů).
- ViewModely pro prezentační logiku (filtrace rozvrhu, náhled/validace, spuštění importu a exportu), navázané na AXAML přes datové vazby a příkazy.

Aplikace využívá asynchronní zpracování (`async/await`) pro čtení souborů, práci s databází a generování PDF, aby GUI zůstalo responzivní. LINQ se používá k projekcím, filtrům a agregacím (týdenní a měsíční součty). Při ukládání a úpravách probíhá kontrola invariantů: bloky v jednom dni se nesmí nelogicky překrývat, celodenní stavy se zapisují jedním záznamem (`AllDay = true`), přestávky respektují nastavené meze a týdenní/měsíční součty odpovídají požadavkům.

Silné typování v C# odhaluje chyby už při kompilaci, knihovny ekosystému **.NET** výrazně zrychlují vývoj a jednotné úložiště *Events* zjednodušuje dotazování i generování výstupů. Díky rozdělení na modely, služby a ViewModely je kód přehledný a připravený na další rozšiřování.

2.1.2 Jazyk XAML

AXAML deklarativně popisuje obrazovky v **Avalonia UI**: průvodce importem **CSV**, kalendářový přehled, editor denních záznamů a náhled před exportem **PDF**. Vizuální část (View) neobsahuje aplikační logiku; veškeré chování je ve **ViewModelech** v C#.

Vazby propojují prvky rozhraní s vlastnostmi ViewModelu (např. kolekce událostí měsíce, agregované součty). Uživatelské akce spouštějí příkazy (***GenerateEPDCommand***, ***CreateEventCommand***, ***ShowPdfPreview***). Změny stavu se okamžitě projeví v UI bez ručního překreslování.

Opakující se části (řádky/denní buňky, značky svátků a absencí) jsou definovány pomocí **DataTemplate** a **ControlTemplate**. **Styly** sjednocují barvy, písma a odsazení tak, aby aplikace měla konzistentní vzhled a bylo snadné jej upravit bez zásahu do C#.

Ve spojení s **ReactiveUI** jsou ViewModely reaktivní: změny dat (např. vložení půldenní dovolené) okamžitě přepočítají týdenní součet a aktualizují náhled tabulky. Tím zůstává rozhraní plynulé i při práci s většími měsíci.

AXAML drží prezentační vrstvu izolovanou, takže se „co se zobrazuje“ (AXAML) a „co se počítá“ (C#) může vyvíjet nezávisle. Zároveň zůstává zachována jednoduchá integrace přes bindingy a příkazy, což zrychluje iterace nad UI i logikou.

2.2 Struktura aplikace

Struktura aplikace je postavena na architektonickém vzoru MVVM, který je pro Avalonia UI přirozenou volbou. Tento vzor umožňuje důsledné oddělení prezentační vrstvy od logiky a poskytuje jasný rámec pro organizaci kódu. Díky tomu lze aplikaci snadno rozšiřovat, testovat a udržovat, aniž by došlo k propojení částí, které by měly zůstat nezávislé.

Model tvoří perzistentní entity mapované pomocí Entity Framework Core na SQLite databázi. Nejvýznamnějšími třídami jsou *Event*, *UserSettings* a *GlobalSettings*, které představují základní datové struktury aplikace. Třída *Event* popisuje konkrétní události, *UserSettings* slouží k uložení individuálních nastavení pro vybraný den a *GlobalSettings* uchovává výchozí pracovní časy, parametry přestávek, informace o vyučujícím a semestrální režimy. O přístup k databázi se stará *AppDbContext*, jenž kromě definice tabulek obsahuje konfiguraci ukládání časových údajů a mapování vztahů mezi entitami.

View představují .axaml soubory, které určují vzhled jednotlivých částí rozhraní. Patří sem například *MainWindow.axaml* s navigací a kalendářem, *DayView.axaml*, *WeekView.axaml* a *MonthView.axaml* pro různé pohledy na rozvrh, dále dialogy jako *CreateEventDialog.axaml* nebo *GlobalUserSettingsView.axaml*. Součástí pohledů jsou i specifická chování (Behaviors), která přidávají interaktivitu – například *ShowCollisionMessageBehavior* zajišťuje zobrazení upozornění na kolize událostí. Díky těmto chováním zůstává kód čistý a opakovaně použitelný. Uživatelské rozhraní je navrženo tak, aby bylo intuitivní: levý panel hlavního okna nabízí přepínání mezi denním, týdenním a měsíčním pohledem, správu nastavení a generování PDF, zatímco pravý panel zobrazuje samotná data kalendáře.

ViewModel vrstva funguje jako prostředník mezi modelem a pohledem. Hlavní třídou je *MainWindowViewModel*, který koordinuje načítání dat a přepínání mezi jednotlivými režimy zobrazení. Samostatné ViewModely pro den, týden a měsíc (*DayViewModel*, *WeekViewModel*, *MonthViewModel*) zajišťují logiku a zpracování dat specifické pro danou kategorii kalendáře. Pro práci s globálním nastavením existuje *GlobalUserSettingsViewModel*, který umožňuje načítat a ukládat pracovní intervaly, parametry přestávek a další údaje. Důležitou roli hraje také *CreateEventDialogViewModel*, jenž se stará o validaci vstupů při vytváření a editaci událostí, a *PdfPreviewViewModel*, který připravuje data pro generování a vyvážení pracovních hodin v

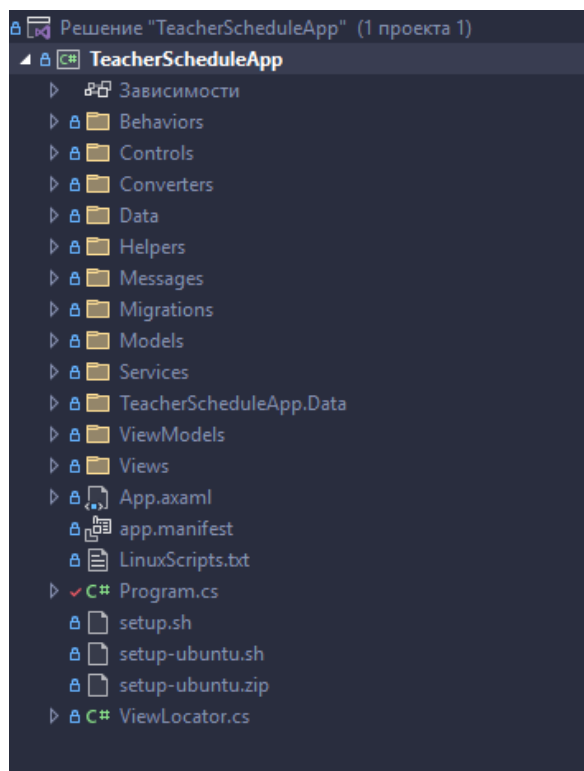
oficiálním PDF. Veškeré změny dat se do rozhraní propisují pomocí mechanismu data bindingu, což znamená, že aktualizace hodnot ve ViewModelu se okamžitě projeví v odpovídajícím pohledu a uživatel vždy pracuje s aktuálními informacemi.

Servisní vrstva doplňuje architekturu o centralizovanou logiku, která by neměla být přímo součástí ViewModelů. Klíčovým prvkem je *EventService*, který kromě klasických CRUD operací implementuje také algoritmus pro vyvažování odpracovaných hodin mezi jednotlivými dny. Tento mechanismus automaticky vyrovnává přebytky a nedostatky v pracovním fondu tak, aby měsíční balance odpovídala nastaveným pravidlům, aniž by uživatel musel data upravovat ručně. *WorkingHoursCalculatorService* zajišťuje výpočty odpracovaného času, *AutomaticEventsGeneratorService* vytváří výchozí události na základě globálních nastavení a rozvrhu, a *PdfService* připravuje výsledné výstupy ve formátu PDF. Zvláštní roli hraje komponenta *EPDGenerator*, která je určena pro načítání rozvrhu vyučujícího ze souboru CSV (exportovaného z IS STAG) a pro převod těchto dat na interní události aplikace. Díky tomu může uživatel rychle naplnit svůj kalendář skutečným rozvrhem a následně využít ostatní nástroje aplikace pro vyvažování, úpravy a export do oficiálních dokumentů. Dále *GlobalSettingsService* a *SettingsService* poskytují rozhraní pro perzistenci uživatelských i globálních konfigurací.

Díky této architektuře je aplikace přehledná, modulární a snadno rozšiřitelná. Každá vrstva má jasně definovanou zodpovědnost: Model obstarává data a jejich uložení, View definuje vzhled a chování uživatelského rozhraní, ViewModel zprostředkovává interakci mezi uživatelem a logikou, a Services centralizují výpočty a komplexní operace. Tento přístup umožnil realizovat robustní desktopovou aplikaci, která je schopná automaticky generovat události, vyrovnávat pracovní fond, zajišťovat multiplatformní běh a poskytovat uživateli komfortní nástroje pro evidenci pracovní doby.

2.4 Souborová struktura aplikace

Struktura projektu je navržena tak, aby jasně oddělovala jednotlivé vrstvy a funkce aplikace a zároveň podporovala zásady přehlednosti, udržitelnosti a rozšiřitelnosti. Díky tomuto členění lze snadno vyhledat části kódu související s databází, prezentační logikou, uživatelským rozhraním či podpůrnými funkcemi. Projekt je rozdělen do samostatných složek, z nichž každá se zaměřuje na konkrétní oblast.



Obrázek 3: Souborová struktura aplikace

Celkově tato souborová struktura poskytuje jasné oddělení zodpovědností, usnadňuje orientaci v projektu a umožňuje efektivní vývoj i budoucí rozšiřování. Každá složka má svou specifickou úlohu, díky čemuž je aplikace modulární a přehledná, a to jak pro jednoho vývojáře, tak i pro týmovou spolupráci.

2.4.1 Složka Behaviors

Složka Behaviors obsahuje třídy využívající Avalonia.Xaml.Behaviors. Jejich cílem je přidat specifické interakce k uživatelským prvkům, aniž by bylo nutné měnit jejich šablony. Nachází se zde například mechanismy pro zachytávání kliknutí na buňky kalendáře či události, komponenty pro zavírání dialogů a potvrzování jejich akcí nebo chování zajišťující zobrazování chybových a kolizních hlášení. Díky tomuto přístupu zůstává kód rozhraní čistý a interakce lze snadno měnit.

2.4.2 Složka Controls

Další částí je složka Controls, která sdružuje vlastní uživatelské prvky rozšiřující základní komponenty Avalonia UI. Tyto prvky slouží k zobrazení událostí v kalendáři nebo k rozvržení panelů a díky své znovupoužitelnosti přispívají k modulárnosti a konzistenci aplikace.

2.4.3 Složka Converters

Transformace dat mezi ViewModely a zobrazením je soustředěna ve složce Converters. Převodníky implementující `IValueConverter` zde slouží k převodu logických hodnot na viditelnost či barvy, k výpočtu pozic událostí v kalendáři nebo k transformaci časových údajů na grafické rozměry. Tento přístup umožňuje udržet čistý MVVM vzor a logiku transformace oddělit od prezentace.

2.4.4 Složka Data

Datovou vrstvu představuje složka Data obsahující kontext `AppDbContext`. Ten definuje přístup k databázi prostřednictvím Entity Framework Core, deklaruje sady entit pro události a nastavení a zajišťuje připojení k databázovému souboru SQLite uloženému v uživatelském adresáři. Současně mapuje vlastnosti na databázové sloupce a obsahuje konvertory pro jednotné ukládání časových hodnot.


```

Ссылка: 42 | Egor Razboinikov, 3 дн. назад | Автор: 1, изменений: 3
public class AppDbContext : DbContext
{
    Ссылка: 45 | Egor Razboinikov, 112 дн. назад | Автор: 1, изменение: 1
    public DbSet<Event> Events { get; set; }
    Ссылка: 6 | Egor Razboinikov, 112 дн. назад | Автор: 1, изменение: 1
    public DbSet<UserSettings> UserSettings { get; set; }
    Ссылка: 4 | Egor Razboinikov, 112 дн. назад | Автор: 1, изменение: 1
    public DbSet<GlobalSettings> GlobalSettings { get; set; }

    Ссылка: 0 | Egor Razboinikov, 112 дн. назад | Автор: 1, изменение: 1
    protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
    {
        string baseDir;
        if (RuntimeInformation.IsOSPlatform(OSPlatform.Windows))
            baseDir = Environment.GetFolderPath(Environment.SpecialFolder.ApplicationData);
        else
            baseDir = Path.Combine(Environment.GetFolderPath(Environment.SpecialFolder.UserProfile), ".config");

        var appDir = Path.Combine(baseDir, "TeacherScheduleApp");
        Directory.CreateDirectory(appDir);

        var dbPath = Path.Combine(appDir, "teacherapp.db");
        optionsBuilder.UseSqlite($"Data Source={dbPath}");
    }

    Ссылка: 0 | Egor Razboinikov, 3 дн. назад | Автор: 1, изменений: 3
    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        var dateTimeConverter = new ValueConverter<DateTime, string>(
            v => v.ToString("yyyy-MM-dd HH:mm:ss", CultureInfo.InvariantCulture),
            v => DateTime.ParseExact(v, "yyyy-MM-dd HH:mm:ss", CultureInfo.InvariantCulture)
        );
        modelBuilder.Entity<Event>(b =>
        {
            b.HasKey(e => e.Id);
            b.Property(e => e.Id).ValueGeneratedOnAdd();
        });
        modelBuilder.Entity<Event>()
            .HasOne<Event>()
            .WithMany()
            .HasForeignKey(e => e.ParentEventId)
            .IsRequired(false)
            .OnDelete(DeleteBehavior.Cascade);

        modelBuilder.Entity<Event>()
            .Property(e => e.StartTime)
            .HasConversion(dateTimeConverter);

        modelBuilder.Entity<Event>()
            .Property(e => e.EndTime)
            .HasConversion(dateTimeConverter);
        base.OnModelCreating(modelBuilder);
    }
}

```

Obrázek 4 Ukázka kódu třídy AppDbContext

2.4.5 Složka Helpers

Podpůrnou funkci plní složka Helpers, kde jsou umístěny univerzální třídy využívané napříč aplikací. Patří sem například logika pro získání reference na hlavní okno nebo komponenta zpracovávající seznam státních svátků prostřednictvím knihovny Nager.Date.

2.4.6 Složka Messages

Komunikaci mezi částmi systému zajišťuje složka Messages. Obsahuje definice zpráv, které informují ViewModely o důležitých změnách, například o vytvoření nových automatických událostí, dokončení generování PDF či úpravě nastavení. Díky tomuto mechanismu spolu moduly spolupracují bez přímých závislostí.

2.4.7 Složka Models

Datové entity jsou definovány ve složce Models. Nachází se zde třídy popisující události, globální i uživatelská nastavení nebo jednotlivé bloky výuky. Jedná se o jednoduché objekty mapované prostřednictvím Entity Frameworku na databázové tabulky.

2.4.8 Složka Services

Složka Services je jádrem obchodní logiky. Obsahuje služby starající se o správu událostí, generování automatických bloků, výpočet pracovních hodin nebo tvorbu PDF souborů. Zvláštní význam má algoritmus vyvažování pracovní doby, který se snaží rozdělit odpracované hodiny tak, aby se měsíční balance co nejvíce blížila očekávané normě. Do této části patří také třída EPDGenerator, která čte rozvrh učitele ze souboru CSV a převádí jej na databázové události.

2.4.9 Složka ViewModels

Prezentační logiku soustřeďuje složka ViewModels. Každý ViewModel propojuje data s konkrétním uživatelským rozhraním a definuje příkazy pro práci s aplikací. Hlavní ViewModel zajišťuje přepínání mezi denním, týdenním a měsíčním režimem, další spravují dialogy, nastavení nebo náhled PDF.

2.4.10 Složka Views

Vizuální vrstvu tvoří složka Views, v níž se nacházejí šablony .axaml a jejich code-behind. Tyto soubory definují vzhled hlavního okna, jednotlivých kalendářních pohledů a dialogů. Jejich oddělení od logiky usnadňuje úpravy uživatelského rozhraní.

2.4.11 Složka Kořenové soubory projektu

Kořenové soubory projektu obsahují deklarace globálních zdrojů a startovací logiku. App.axaml definuje styly a šablony, jeho code-behind inicializuje hlavní okno a životní cyklus aplikace. Manifest určuje metadata pro Windows, zatímco Program.cs představuje vstupní bod aplikace. ViewLocator pak zajišťuje propojení ViewModelů s odpovídajícími Views. Celkově je struktura navržena tak, aby byla modulární, přehledná a snadno rozšiřitelná.

2.5 Struktura databáze

Aplikace používá SQLite jako lokální úložiště v jednom souboru a **Entity Framework Core (EF Core)** pro mapování entit a migrace. Schéma je záměrně jednoduché a **neobsahuje relační vazby na úrovni databáze** (tj. nejsou definovány cizí klíče ani referenční omezení).

2.5.1 Přehled tabulek

Events – jednotná tabulka pro **všechny události dne, které zapisuje uživatel nebo je vytvoří aplikace** (např. automatický blok přestávky). Neobsahuje žádné systémové záznamy pro víkendy nebo svátky:

- **Id** – primární klíč (PK).
- **StartTime, EndTime** – začátek/konec bloku (časové úseky dne).
- **EventType** – typ události (práce, D, N, OČR, L, PC, S).
- **Title, Description** – volitelné popisky.
- **AllDay** – je-li záznam celodenní (např. dovolená na celý den).
- **ArrivalTime, DepartureTime, LunchStart, LunchEnd** – reálné hrany dne a polední přestávka pro daný den (pokud je relevantní).
- **IsAutoGenerated** – automaticky vložený blok (např. přestávka dopočítaná algoritmem).
- **ParentEventId** – identifikace zdrojové události pro blok (logická reference; bez cizího klíče).
- **HasCollision** – příznak zjištěného překryvu dvou stavů.
- **ImportBatchId, ImportLabel** – původ dat (šarže importu).
- **AutoGeneratedForDate** – datum, pro které byl auto-blok vytvořen.
- **IsDeleted** – měkké smazání (záznam zůstává v historii, ale běžné dotazy ho neberou v potaz).

GlobalSettings – tabulka globálních pravidel a výchozích hodnot (pro rok/semestr):

- identifikace vyučujícího (jméno, útvar),
- výchozí okno dne (globální příchod/odchod),
- denní defaulty (pondělí–pátek: příchod, odchod, oběd **LunchStart/LunchEnd**),
- limity přestávky (**MinBreakDuration, MaxBreakDuration**),
- názvy automatických bloků (před/po obědě).

UserSettings – tabulka uživatelských přenastavení pro konkrétní kalendářní dny:

- *Id (PK), Date (den),*
- *ArrivalTime, DepartureTime, LunchStart, LunchEnd.*
- Existuje-li záznam pro daný den, přebíje výchozí hodnoty z *GlobalSettings*.

_EFMigrationsHistory – technická tabulka EF Core s historií migrací schématu.

2.5.2 Bez cizích klíčů: záměr a důvody

Relační **vazby nejsou uplatněny** na úrovni databáze (SQLite **foreign_keys** nejsou vyžadovány). Důvody:

- **Jednoduchost a přenositelnost souboru.**
SQLite je souborová databáze. Udržovat ji „štíhlou“ usnadňuje zálohování, kopírování mezi stroji a případné ruční zásahy při testování.
- **Tolerantní import po šaržích.**
Import z CSV probíhá po dávkách (*ImportBatchId*). Dočasně mohou vzniknout neúplné či „čekající“ záznamy (např. auto-bloky přidáné až po validaci). Absence cizích klíčů zjednodušuje do-import a následné dorovnání bez nutnosti pořadí insertů a bez selhání transakce kvůli referencím.
- **Logika a pravidla jsou v aplikaci**
Všechny invarianty (překryvy, priority stavů, korektní měsíční/týdenní součty) i „logické vazby“ (např. *ParentEventId*) hlídá servisní vrstva a validace při ukládání. Pravidla EPD se tak dají verzovat a měnit v kódu, nikoli v triggerch databáze.
- **Výkon a jednoduché dotazy.**
Typické dotazy se opírají o *StartTime* a filtrování období; nepotřebujeme joiny přes FK. Indexy nad časem a šaržemi dávají větší reálný přínos než formální referenční omezení.

2.5.3 Shrnutí rozhodnutí

Žádné cizí klíče a bez uložených víkendů/svátků menší, odolnější soubor DB, jednodušší importy, snadné vracení šarží a levnější změny pravidel.

Aplikační logika nese odpovědnost za integritu: validace při importu/úpravách, jednotná mřížka času, pravidla EPD, označování kolizí.

Jednotná tabulka Events zjednodušuje dotazování a export, GlobalSettings a UserSettings pokrývají „výchozí vs. výjimky“ bez nutnosti relačních vazeb.

2.6 Uživatelské rozhraní aplikace

Uživatelské rozhraní aplikace je někdy nejdůležitějším aspektem, který určuje, jak snadná a intuitivní bude pro uživatele interakce s aplikací. Jednoduchý a přímočarý návrh uživatelského rozhraní zajišťuje, že aplikace je intuitivní, snadno se používá. To pomáhá uživateli rychle a snadno pochopit, jak aplikace funguje, a může efektivně využívat její funkce. Význam uživatelského rozhraní by se neměl podceňovat. Koneckonců je to tvář aplikace a měla by být atraktivní. Pokud rozhraní obsahuje velké množství prvků nebo je obtížné se v něm orientovat, může to mít negativní dopad na uživatelský zážitek. Naopak dobře navržené uživatelské rozhraní s jasným vizuálem, a nejen strukturou může pro uživatele hrát velmi velkou roli. Klíčovými prvky dobrého uživatelského rozhraní jsou jednoduchost, přehlednost a konzistence.

2.6.1 Hlavní stránka aplikace

První věc, kterou uživatel uvidí po spuštění aplikace, je její hlavní stránka. Při prvním spuštění aplikace se také automaticky vytvoří události na celý rok. Hlavní stránka je rozdělena na dva hlavní panely. Levý panel slouží k různým interakcím s aplikací. Druhý panel slouží k zobrazení pohledů na daný den, týden nebo měsíc v závislosti na tom, co uživatel vybral v rozevíracím seznamu na levém panelu. Výchozí zobrazení je aktuální den.

Popíšeme podrobně levý panel. V horní části tohoto panelu jsou čtyři tlačítka, která slouží k vyvolání různých funkcí. První tlačítko „Vytvořit událost“ slouží k vyvolání dialogového okna (jeho popis je uveden níže). Druhé tlačítko „Generovat EPD“ slouží k vyvolání dialogového okna pro načtení souboru .csv s rozvrhem učitele ze STAG. Po výběru souboru se spustí proces vytváření událostí ze souboru. Třetí tlačítko „Globální nastavení“ slouží k otevření okna globálních nastavení (podrobný popis níže). Čtvrté tlačítko „Příklad PDF“ slouží k otevření dialogového okna pro prohlížení dokumentu ve formátu PDF. Za tlačítky se nachází kalendář s možností výběru dne, měsíce a roku. Pokud v tomto kalendáři kliknete na jakýkoli den, aplikace přejde a zobrazí údaje pro tento den. Dále je zde rozevírací prvek, který zobrazuje informace o nahraných souborech. Za ním se nachází rozevírací seznam, který slouží k výběru možnosti zobrazení dne, týdne nebo měsíce. Za seznamem následuje blok nastavení pro konkrétní den. Tam se nacházejí 4 hlavní pole, která slouží k zadání času příchodu, času

odchodu a začátku a konce přestávky, a tlačítko, které uloží zadané informace. Stojí za zmínku, že tento blok je viditelný pouze v denním režimu. Za tímto blokem se zobrazují informace o tom, kolik hodin bylo odpracováno z normy.

Vytvořit událost

Načíst rozvrh

Globální nastavení

EPD náhled

srpen 2025

^

▼

P Ú S Č P S N

28	29	30	31	1	2	3
4	5	6	7	8	9	10
11	12	13	14	15	16	17
18	19	20	21	22	23	24
25	26	27	28	29	30	31
1	2	3	4	5	6	7

Načtený soubory

▼

Rozvrh

▼

Nastavení pro den:

25.08.2025

Příchod:

08:30

Odchod:

17:00

Oběd od:

12:30

Oběd do:

13:00

Uložit nastavení

Obrázek 5 Levý panel aplikace

← Den: 25 srp 2025 → Dnes	
04:00	
05:00	
06:00	
07:00	
08:00	● Dopolední pracovní doba
09:00	
10:00	
11:00	
12:00	
13:00	● Oběd ● Odpolední pracovní doba
14:00	
15:00	
16:00	
17:00	
18:00	

Obrázek 6 Pravý panel aplikace

2.6.2 Dialogové okno pro vytvoření události

Dialogové okno obsahuje následující prvky. První pole slouží k zadání nebo úpravě názvu události. Toto pole je povinné, při pokusu o vytvoření události bez názvu se zobrazí dialogové okno s chybou. Druhé pole již není povinné a slouží k přidání popisu události. Poté následuje tlačítko, které určuje, zda bude událost trvat celý den. Poté následuje blok pro nastavení data a času události. Za blokem pro nastavení data je prvek, který umožňuje vybrat typ události (pracovní, volno, služební cesta atd.). A posledními prvky jsou tlačítko pro zrušení, které zavře okno, a tlačítko pro uložení, které událost vytvoří a okno zavře.

Nová událost

Název

Popis

☐ Celý den

Začátek:

4 květen 2025 0 00

Konec:

4 květen 2025 0 00

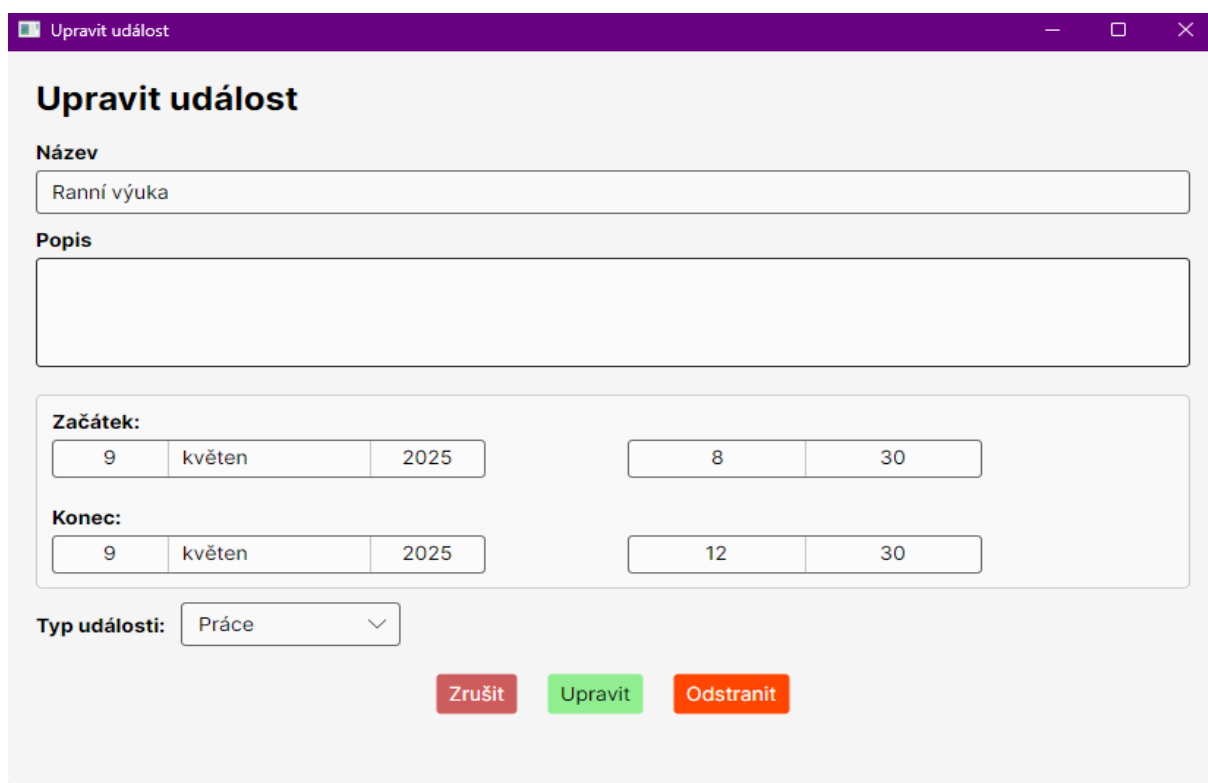
Typ události: Práce

Zrušit Vytvořit

Obrázek 7 Dialogové okno pro vytvoření

2.6.3 Dialogové okno pro změnu události

Dialogové okno pro změnu události obsahuje následující prvky. První pole slouží k zadání nebo změně názvu události. Toto pole je povinné, při pokusu o změnu události bez názvu se zobrazí dialogové okno s chybou. Druhé pole není povinné a slouží k přidání popisu události. Dále následuje blok pro nastavení data a času události. Za blokem pro nastavení data se nachází prvek, který umožňuje vybrat typ události (pracovní, volno, služební cesta atd.). Posledními prvky jsou tlačítka pro zrušení, které zavře okno, tlačítko pro uložení změn, které aktualizuje událost a zavře okno, a tlačítko, které slouží k odstranění události. Před odstraněním události bude uživatel požádán o potvrzení odstranění události.



Upravit událost

Název

Ranní výuka

Popis

Začátek:

9 květen 2025 8 30

Konec:

9 květen 2025 12 30

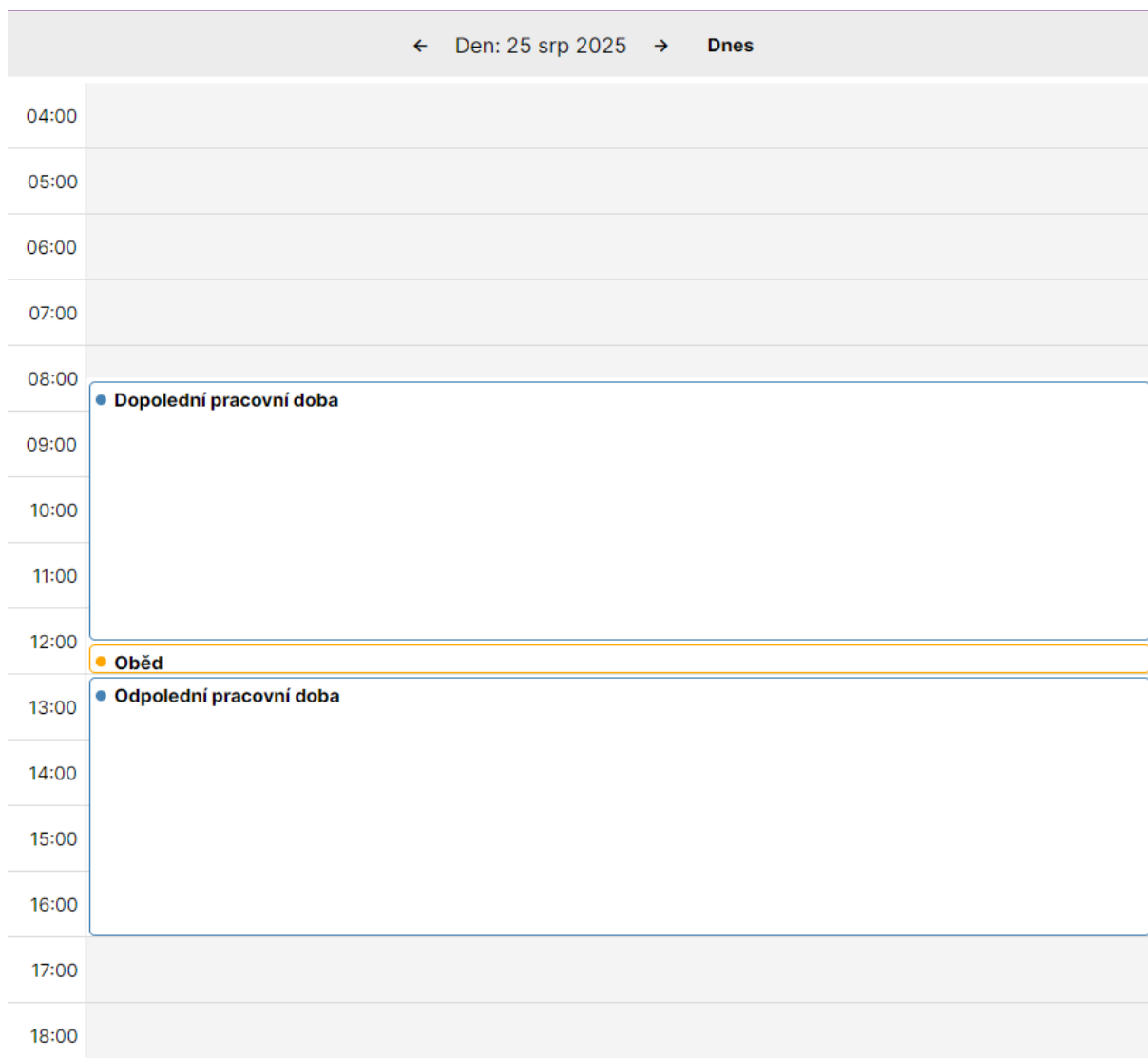
Typ události: Práce

Zrušit Upravit Odstranit

Obrázek 8 Dialogové okno pro změnu

2.6.4 Vizuální prezentace pro den

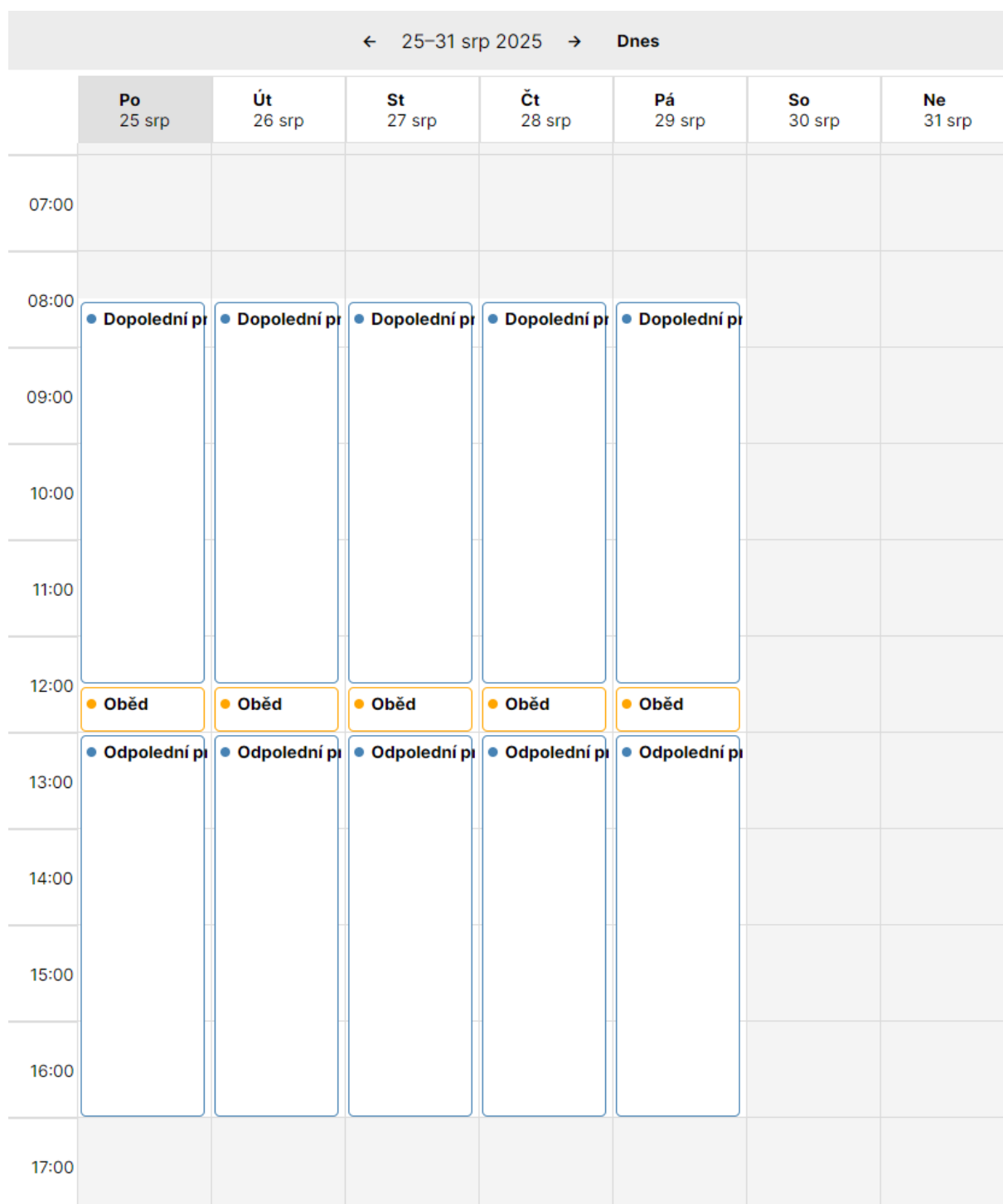
Vizuální prezentace pro daný den obsahuje mřížku s hodinami, pracovní interval je barevně odlišen od nepracovního. V horní části jsou tlačítka, která umožňují přepínat mezi dny. Tam se také nachází tlačítko, které přesune uživatele na aktuální den. Stejně tak zobrazuje události pro daný den, pokud nějaké existují.



Obrázek 9 Vizuální prezentace pro den

2.6.5 Vizuální prezentace pro týden

Vizuální prezentace pro týden obsahuje také tabulku s hodinami a dny, které tvoří týden. V horní části jsou tlačítka, která umožňují přepínat mezi týdny. Tam se také nachází tlačítko, které přesune uživatele na aktuální týden. Stejně tak zobrazuje události pro tento týden, pokud nějaké jsou.



Obrázek 10 Vizuální prezentace pro týden

2.6.6 Vizuální prezentace pro měsíc

Vizuální prezentace pro měsíc obsahuje také mřížku, ale již s dny, které spadají do tohoto měsíce. Státní svátky, víkendy a dny z předchozího nebo následujícího měsíce se liší barvou pozadí nebo barvou čísla dne. V horní části jsou tlačítka, která umožňují přepínat mezi měsíci. Tam se také nachází tlačítko, které přesune uživatele na aktuální měsíc. Stejně tak zobrazuje události pro tento měsíc, pokud nějaké jsou.

← srpen 2025 → Dnes						
Po	Út	St	Čt	Pá	So	Ne
28 ● 08:30 Dopol ● 12:30 Oběd	29 ● 08:30 Dopol ● 12:30 Oběd	30 ● 08:30 Dopol ● 12:30 Oběd	31 ● 08:30 Dopol ● 12:30 Oběd	1 ● 08:30 Dopol ● 12:30 Oběd	2	3
4 ● 08:30 Dopol ● 12:30 Oběd	5 ● 08:30 Dopol ● 12:30 Oběd	6 ● 08:30 oo	7 ● 08:30 Dopol ● 12:30 Oběd	8 ● 08:30 Dopol ● 12:30 Oběd	9	10
11 ● 08:30 Dopol ● 12:30 Oběd	12 ● 08:30 Dopol ● 12:30 Oběd	13 ● 08:30 Dopol ● 12:30 Oběd	14 ● 08:30 Dopol ● 12:30 Oběd	15 ● 08:30 Dopol ● 12:30 Oběd	16	17
18 ● 08:30 Dopol ● 12:30 Oběd	19 ● 08:30 Dopol ● 12:30 Oběd	20 ● 08:30 Dopol ● 12:30 Oběd	21 ● 08:30 Dopol ● 12:30 Oběd	22 ● 08:30 Dopol ● 12:30 Oběd	23	24
25 ● 08:30 Dopol ● 12:30 Oběd	26 ● 08:30 Dopol ● 12:30 Oběd	27 ● 08:30 Dopol ● 12:30 Oběd	28 ● 08:30 Dopol ● 12:30 Oběd	29 ● 08:30 Dopol ● 12:30 Oběd	30	31
1 ● 08:00 Dopol ● 12:00 Oběd	2 ● 08:00 Dopol ● 12:00 Oběd	3 ● 08:00 Dopol ● 12:00 Oběd	4 ● 08:00 Dopol ● 12:00 Oběd	5 ● 08:00 Dopol ● 12:00 Oběd	6	7

Obrázek 11 Vizualní prezentace pro měsíc

2.6.7 Globální nastavení

Globální nastavení obsahuje velké množství polí pro zadání informací o různých časových intervalech a dalších informací o uživateli, které budou použity pro vytváření automatických událostí, generování PDF a tak dále. Podrobný popis začneme tlačítky, která slouží k přepínání mezi zimním a letním semestrem, což uživateli umožňuje zadat různé údaje pro konkrétní semestr. Pole pro zadávání informací jsou stejná pro obě varianty. Dále následuje blok, který

odpovídá za globální interval pracovní doby. Poté následuje blok s možností zadání informací o příchodu do práce a odchodu z práce a také času přestávky. Tato pole jsou stejná pro všechny pracovní dny. Zložka další nastavení obsahuje pole pro zadání jména, katedry, které budou použity pro generování v PDF. Pole pro zadání omezení pro přestávku a také jména, která se používají pro vytváření automatických událostí. Na konci se také nachází tlačítko pro uložení.

[← Zpět ke kalendáři](#)**Rok 2025 · Letní semestr**Rok:

2025

☐ Zimní ☒ Letní

Globální pracovní doba · Letní semestr

Od:

08:30

Do:

17:00

Pracovní doba · Letní semestr

Pondělí Úterý Středa Čtvrtek Pátek

Příchod:

08:30

Odchod:

17:00

Oběd, začátek:

12:30

Oběd, konec:

13:00

Další nastavení

Údaje pro PDF

Jméno:

Radek Matoušek

Útvar:

Katedra informačních technologií

Pauzy

Min. délka pauzy:

00:15

Max. délka pauzy:

01:00

Názvy událostí

Před obědem:

Dopolední pracovní doba

Pro oběd:

Oběd

Po obědě:

Odpolední pracovní doba

Uložit nastavení

Obrázek 12 Globální nastavení

2.6.8 Ukázka PDF

Náhled PDF obsahuje dva prvky, se kterými lze interagovat. První slouží k přepínání mezi měsíci, druhý k uložení souboru do počítače. Také obsahuje obrázek, jak bude stažený soubor vypadat.

07-2025

Uložit PDF

Evidence pracovní doby, včetně přestávek v práci a práce přesčas

Fakulta elektrotechniky a informatiky
jméno: Radek Matoušek
útvár: Katedra informačních technologií

pracovní doba: 08:30–17:00 hod.
docházka za měsíc: červenec 2025

Fond prac. doby:
184 hodin

Den	Začátek pracovní doby	1. přestávka		2. přestávka		Konec pracovní doby	Odpracováno	Přesčas	Neodprac.	Poznámka)
		Od	Do	Od	Do					
1.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
2.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
3.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
4.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
5.										S
6.										S
7.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
8.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
9.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
10.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
11.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
12.										
13.										
14.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
15.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
16.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
17.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
18.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
19.										
20.										
21.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
22.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
23.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
24.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
25.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
26.										
27.										
28.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
29.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
30.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
31.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
Celkem							184:00:00	00:00:00	00:00:00	

kontr. č. Σ odprac. - přesčas + neodprac. hodin (odpovídá FPD): 184:00:00

podpis zaměstnance:

podpis nadřízeného pracovníka:

*) do poznámky: D – dovolená, N – nemoc, OČR – ošetřování, L – lékař, PC – pracovní cesta, S – svátek a ostatní dny pracovního klidu

Obrázek 13 Ukázka PDF

2.7 Algoritmy

Tato kapitola shrnuje implementované algoritmy, které zajišťují praktické fungování aplikace: jednak vyrovnávání hodin podle pravidel EPD jednak import rozvrhu ze souboru CSV exportovaného ze systému STAG. Popis odpovídá aktuálním třídám a metodám v projektu.

2.7.1 Algoritmus vyrovnávání hodin

Z dostupných záznamů dne vytvořit takové rozložení práce, aby **každý pracovní týden** vyhověl požadované hodnotě a zároveň byl zachován **měsíční fond**. Úpravy probíhají v povolených mezích: přidávají se nebo zkracují **automatické pracovní úseky** (auto-bloky) a upravuje se **oběd** tak, aby ležel uvnitř pracovního okna dne. Víkendy a státní svátky se **neukládají** do databáze – berou se z kalendářní služby při výpočtech.

Pro každý den se nejprve vypočtou metriky pomocí *WorkingHoursCalculatorService.DailyMetrics(...)*.

V interní reprezentaci rozlišujeme:

- **workInclBT** – součet hodin práce včetně pracovních cest (PC – pracovní cesta),
- **specialNonPc** – součet hodin zvláštních stavů (D, N, OČR, L) mimo PC.

Denní „odečitatelné“ stavy (D/N/OČR/L) mohou časově **překrývat** výuku; ve výpočtu mají přednost a do práce se v překryvu nezapočítají. Naopak PC se počítá jako práce.

Vyrovnaní v měsíci vstupem je rok a měsíc; metoda **BalanceEventsForMonthAsync(year, month, ...)**:

1. Vytvoří **seznam pracovních dnů** měsíce (bez víkendů a svátků).
2. Rozdělí je do **ISO týdnů** (stejně, jako v kalendáři ISO 8601).
3. Každý týden předá do **BalanceOneWeekAsync(...)**.
4. Pokud je **první i poslední týden** v měsíci neúplný (přesah přes hranici měsíce), provede ještě **druhé kolo vyrovnaní mezi těmito dvěma týdny** (**BalanceTwoWeeksAsync(...)**).
5. Na závěr spustí **PostNormalizeMonthAsync(...)** (dočištění denních detailů – oběd, drobné doladění auto-bloků, viz níže).

Vyrovnaní v týdnu

BalanceOneWeekAsync(weekDays) dělá čistě **praktický přesun hodin**:

- Pro každý den spočítá **Total = specialNonPc + workInclBT** a od něj odvodí:
 - **Need** (kolik chybí do osmi hodin),
 - **Extra** (o kolik je přes osm hodin).
- Vzniknou tři fronty: **deficity**, **excesy** a **flexibilní dny** (dny s ~8 h, ale s možností zkracovat/rozšiřovat okraje, tj. mají auto-blok na začátku nebo konci).
- Během smyčky:
- z „dárce“ s Extra se odebírá čas **zkrácením auto-bloku** na začátku/konce dne (**TrimStartAuto(...)**, **TrimEndAuto(...)**),
 - totéž množství se **přidá příjemci** (**ExtendStartAuto(...)**, **ExtendEndAuto(...)**),

- po každé změně se přegenerují **automatické události** pro dotčené dny (*AutomaticEventsGeneratorService.RegenerateDailyEventsAsync(...)*), aby seděly přestávky a hrany dne.
- Když dojdou párovatelné dvojice deficit/exces, zkusí se využít **flexibilní dny** (přenést zbylý „nadčas“ do dne, který lze jemně prodloužit).

Důležité detaily implementace

- „Okrajové“ zkracování/rozšiřování dne je možné jen tehdy, když je **okraj tvořen auto-blokem** (tj. úsekem, který může algoritmus bezpečně měnit). Kontrolují metody *CanTrimStart(...)*, *CanTrimEnd(...)*.
- Při změně okraje se vždy **aktualizují uživatelská nastavení dne** (příchod/odchod a oběd) přes *SettingsService.SaveUserSettingsForDate(...)* a vymění se případná událost oběda (*ReplaceLunchEvent(...)*).
- Pokud se oběd ocitl mimo pracovní okno, použije se *EnsureLunchInsideWorkWindowAsync(...)*, případně se oběd „vyřeže“ z vnitřního auto-bloku (*TryCarveLunchFromInnerAuto(...)*), aby vznikl volný interval.

Post-normalizace dne

Metoda *PostNormalizeMonthAsync(...)* ještě po týdenním kole:

- doplní chybějící minuty do osmi hodin, pokud to dává smysl a jsou k dispozici **okrajové auto-bloky**,
- **vynutí oběd** uvnitř pracovního okna dne (příchod–odchod); pokud se nevejde, pokusí se jej vytvořit vyříznutím části vnitřního auto-bloku práce,
- znovu přegeneruje auto-události a uloží konzistentní **UserSettings**.

Omezování denního „přes-osmi“

Speciální „bezbolestná“ verze zkrácení pro jeden den je *TrimOvertimeByAutoBlocksAsync(day)*.

Pokud je total > 8 h, zkusí nejdříve **zkrátit poslední** auto-blok práce; když to nejde, zkrátí **první**. Vždy upraví i příchod/odchod a oběd v **UserSettings**, aby zůstal zápis čitelný a platný.

2.6.2 Algoritmus importu rozvrhu

Import rozvrhu je implementován ve třídě EPDGenerator metodou GenerateEPDEventsAsync(csvPath). Cílem je:

1. přečíst soubor CSV ze STAG,
2. **měkce smazat** staré importy v dotčeném intervalu,
3. vytvořit **nové záznamy** do Events,
4. pro dotčené dny vygenerovat automatické bloky (přestávky, okraje) a
5. omezit případné denní „přes-osmi“.

Kroky importu

1) Načtení a zjištění rozsahu importu

Soubor se čte jako Windows-1250 (Encoding.GetEncoding("windows-1250")). Nejprve projdeme řádky, abychom z polí (sloupce 43 a 44) zjistili **nejranější a nejpozdější datum** importu (importStart, importEnd). Pokud je rozsah validní, zavoláme BulkSoftDeleteImportedInRangeAsync(importStart, importEnd), což **měkce smaže** (IsDeleted = true) všechny dříve importované záznamy v daném období.

2) Identifikace šarže

Vygeneruje se batchId (GUID bez pomlček) a batchName (název CSV). Těmito údaji se označí všechny nově vzniklé události kvůli auditu a snadným revertům.

3) Parsování řádků

Každý řádek CSV (oddělovač ;) se rozseká a ošetří oříznutím uvozovek. Důležité sloupce, jak je využívá kód:

- **čas začátku/konce:** indexy **30** a **31**,
- **rozsah týdnů:** **32** (od) a **33** (do),
- **datum základní a od-do:** **42** (base) a **43–44** (dateFrom–dateTo),
- **parita:** **35** (znak L/S/J/K),
- texty pro **Title** a **Description**: např. **3**, **20**, **15**, **16**.

Pro každý kalendářní den v intervalu *dateFrom..dateTo*, který má stejný **den v týdnu** jako *baseDate*, se vypočte **ISO týden** a projde se **paritní filtr**:

- **L** (liché týdny): propouští pouze týdny s *isoWeek % 2 == 1*,
- **S** (sudé týdny): propouští pouze týdny s *isoWeek % 2 == 0*,
- **J**: „každý druhý týden“ s posunem od *weekFrom* (v kódu $((isoWeek - weekFrom) \% 2) != 0$),
- **K** (default): bez omezení parity.

Pokud řádek vyhoví, přidá se do **kolekce nových událostí** jako Event typu **Work** s časem *StartTime.EndTime a* s označením *ImportBatchId/ImportLabel*.

4) Hromadné vložení a návazné generování

CreateEventsBulk(...) vloží záznamy do Events. Z **množiny dotčených dnů** se pro každý den provede:

- *AutomaticEventsGeneratorService.RegenerateDailyEventsAsync(day)* – doplní auto-bloky (okraje, přestávku) v souladu s výchozími pravidly,
- *AdjustUserSettingsForDay(day)* – nastaví **příchod** na první blok práce, **odchod** na poslední blok a **oběd** podle existující události nebo výchozích hodnot,
- *TrimOvertimeByAutoBlocksAsync(day)* – případně zkrátí auto-bloky, aby denní součet nepřesahoval osm hodin.

5) Signály do UI

Na konci importu se přes *MessageBus* vyšlou zprávy *AutoEventsGeneratedMessage* a *EpdGeneratedMessage*, aby **GUI** obnovilo přehledy a souhrny.

Chování vůči víkendům a svátkům

Importer **nevrhává** žádné systémové záznamy pro víkendy a svátky. Pokud rozvrh obsahuje výjimku (např. pracovní událost v sobotu), uloží se jako běžná Event; jinak se víkendy/svátky **neperzistují** a berou se z kalendářní služby až při výpočtech a ve výstupu.

ZÁVĚR

Cílem této bakalářské práce bylo navrhnout, implementovat a ověřit multiplatformní desktopovou aplikaci pro digitalizaci evidence pracovní doby vyučujících, která automatizuje zpracování vstupních dat (rozvrh ze STAG, přestávky, školní prázdniny a státní svátky) a generuje validní PDF dokument připravený k odevzdání. V teoretické části práce jsme analyzovali klíčové technologie a návrhové vzory využívané při vývoji aplikace. Popisovali jsme programovací jazyk C#, relační databázi SQLite a deklarativní značkovací jazyk .axaml v Avalonia UI. Dále jsme představili vývojové prostředí Microsoft Visual Studio a SQLiteStudio a přiblížili architekturu desktopové aplikace založenou na MVVM. Podrobně jsme rozebrali využívané NuGet balíčky (Avalonia, ReactiveUI, EF Core, QuestPDF a další), které zajišťují robustnost, přenositelnost a reaktivitu řešení.

V praktické části práce jsme detailně popsali strukturu SQLite databáze se třemi tabulkami (Events, GlobalSettings, UserSettings), uspořádání projektové složkové struktury dle principu Separation of Concerns a implementaci prezentační vrstvy v Avalonia UI. Klíčovou součástí byla také realizace algoritmu pro vyvážení odpracovaných hodin, který se spouští při generování náhledu PDF a automaticky upravuje časy událostí tak, aby odpovídaly globálním či uživatelským nastavením.

Výsledná aplikace poskytuje uživatelsky přívětivé rozhraní pro rychlé zadávání a úpravy dat, vizualizaci denního, týdenního i měsíčního kalendáře a okamžitý náhled výsledného PDF dokumentu. Díky použití Avalonia UI běží plynule jak na Windows, tak na Linuxu, a díky Entity Framework Core se SQLite nabízí spolehlivé a přenosné datové úložiště.

Navržené řešení významně zkracuje čas potřebný k přípravě měsíčních a týdenních vyúčtování pracovní doby a minimalizuje riziko chyb při manuálním přepisování dat. Aplikace tak přispívá ke zvýšení administrativní efektivity a transparentnosti správy pracovního fondu na akademických pracovištích. V budoucnu je možné řešení rozšířit o jednotnou autentizaci (SSO), auditní logování či export do dalších formátů, což dále posílí jeho použitelnost a škálovatelnost.

POUŽITÁ LITERATURA

1. **MICROSOFT**, 2025. *A tour of the C# language*. In: **learn.microsoft.com** [online]. Microsoft, 2025 [cit. 2025-08-24]. Dostupné z: <https://learn.microsoft.com/cs-cz/dotnet/csharp/tour-of-csharp/overview>
2. **MICROSOFT**, 2025. *.NET – Build modern apps and services* (úvod, multiplatformnost). In: **dotnet.microsoft.com** [online]. Microsoft, 2025 [cit. 2025-08-24]. Dostupné z: <https://dotnet.microsoft.com/en-us/>
3. **AVALONIA UI**, 2025. *The MVVM Pattern*. In: **docs.avaloniaui.net** [online]. Avalonia, 2025 [cit. 2025-08-24]. Dostupné z: <https://docs.avaloniaui.net/docs/concepts/the-mvvm-pattern/>
4. **REACTIVEUI COMMUNITY**, 2025. *Getting Started*. In: **reactiveui.net** [online]. ReactiveUI, 2025 [cit. 2025-08-24]. Dostupné z: <https://www.reactiveui.net/docs/getting-started/>
5. **SQLITE**, 2024. *SQL As Understood By SQLite*. In: **sqlite.org** [online]. SQLite, 2024 [cit. 2025-08-24]. Dostupné z: <https://www.sqlite.org/lang.html>
6. **QUESTPDF**, 2025. *QuestPDF – C# PDF Generation Library*. In: **questpdf.com** [online]. QuestPDF, 2025 [cit. 2025-08-24]. Dostupné z: <https://www.questpdf.com/>
7. **ELECTRON**, 2025. *What is Electron?* In: **electronjs.org/docs/latest** [online]. Electron, 2025 [cit. 2025-08-24]. Dostupné z: <https://electronjs.org/docs/latest/>
8. **PYTHON SOFTWARE FOUNDATION**, 2025. *Python 3.13 documentation*. In: **docs.python.org** [online]. PSF, 2025 [cit. 2025-08-24]. Dostupné z: <https://docs.python.org/>
9. **THE QT COMPANY**, 2025. *Qt for Python (PySide6) – Official documentation*. In: **doc.qt.io** [online]. Qt, 2025 [cit. 2025-08-24]. Dostupné z: <https://doc.qt.io/qtforpython-6/>
10. **JETBRAINS**, 2025. *Compose Multiplatform – Get Started*. In: **jetbrains.com** [online]. JetBrains, 2025 [cit. 2025-08-24]. Dostupné z: <https://www.jetbrains.com/help/kotlin-multiplatform-dev/compose-multiplatform.html>
11. **MICROSOFT**, 2023. *ASP.NET MVC Overview – Model–View–Controller*. In: **learn.microsoft.com** [online]. Microsoft, 2023 [cit. 2025-08-24]. Dostupné z: <https://learn.microsoft.com/en-us/aspnet/mvc/overview/older-versions-1/overview/asp-net-mvc-overview>

12. **REACTIVEUI COMMUNITY**, 2025. *The Zen of the ViewModel*. In: **reactiveui.net** [online]. ReactiveUI, 2025 [cit. 2025-08-24]. Dostupné z: <https://www.reactiveui.net/docs/handbook/view-models/>
13. **CMU/SEI**, 2010. *What Is Your Definition of Software Architecture?* In: **sei.cmu.edu** [online]. Carnegie Mellon University, 2010 [cit. 2025-08-24]. Dostupné z: <https://www.sei.cmu.edu/library/what-is-your-definition-of-software-architecture/>
14. **MICROSOFT**, 2023. *Data binding a chování v Avalonia a WPF (Behaviors)*. In: **learn.microsoft.com** [online]. Microsoft, 2023 [cit. 2025-08-24]. Dostupné z: <https://learn.microsoft.com/cs-cz/dotnet/desktop/wpf/advanced/behaviors-overview>
15. **AVALONIA XAML BEHAVIORS**, 2024. *GitHub – Avalonia.Xaml.Behaviors*. In: **github.com/AvaloniaXAMLBehaviors** [online]. Avalonia Team, 2024 [cit. 2025-08-24]. Dostupné z: <https://github.com/AvaloniaXAMLBehaviors>
16. **AVALONIA COMMUNITY**, 2024. *Behaviors v Avalonia UI*. In: **docs.avaloniaui.net** [online]. Avalonia Community, 2024 [cit. 2025-08-24]. Dostupné z: <https://docs.avaloniaui.net/docs/controls/behaviors>
17. **MICROSOFT**, 2023. *Common web application architectures – monolitická aplikace*. In: **learn.microsoft.com** [online]. Microsoft, 2023 [cit. 2025-08-24]. Dostupné z: <https://learn.microsoft.com/en-us/dotnet/architecture/modern-web-apps-azure/common-web-application-architectures>
18. **MICROSOFT**, 2025. *Microservices architecture style – přehled a trade-offy*. In: **learn.microsoft.com** [online]. Microsoft, 2025 [cit. 2025-08-24]. Dostupné z: <https://learn.microsoft.com/en-us/azure/architecture/guide/architecture-styles/microservices>
19. **FOWLER, Martin**, 2015. *Monolith First*. In: **martinfowler.com** [online]. 2015 [cit. 2025-08-24]. Dostupné z: <https://martinfowler.com/bliki/MonolithFirst.html>
20. **AZURE ARCHITECTURE CENTER**, 2025. *Architecture styles – přehled*. In: **learn.microsoft.com** [online]. Microsoft, 2025 [cit. 2025-08-24]. Dostupné z: <https://learn.microsoft.com/cs-cz/azure/architecture/guide/architecture-styles/>
21. **MICROSOFT**, 2024. *Entity Framework Core – SQLite provider*. In: **learn.microsoft.com** [online]. Microsoft, 2024 [cit. 2025-08-24]. Dostupné z: <https://learn.microsoft.com/cs-cz/ef/core/providers/sqlite/>
22. **ARTIFEX SOFTWARE**, 2025. *Ghostscript – official documentation*. In: **ghostscript.com** [online]. Artifex Software, 2025 [cit. 2025-08-24]. Dostupné z: <https://www.ghostscript.com/>

23. **HABJAN, John** a kol., 2025. *Ghostscript.NET – .NET wrapper pro Ghostscript*. In: **github.com/jhabjan/Ghostscript.NET** [online]. 2025 [cit. 2025-08-24]. Dostupné z: <https://github.com/jhabjan/Ghostscript.NET>
24. **MICROSOFT**, 2025. *Visual Studio – Product documentation (Overview, IDE features)*. In: **learn.microsoft.com** [online]. Microsoft, 2025 [cit. 2025-08-24]. Dostupné z: <https://learn.microsoft.com/cs-cz/visualstudio/ide/?view=vs-2022>
25. **SQLTESTUDIO**, 2025. *SQLiteStudio – Official website (Features, Downloads, Docs)*. In: **sqlitestudio.pl** [online]. 2025 [cit. 2025-08-24]. Dostupné z: <https://sqlitestudio.pl/>

SEZNAM PŘÍLOH

Příloha A: Zdrojový kód aplikace TeacherScheduleApp

PŘÍLOHA A: Zdrojový kód aplikace TeacherScheduleApp

Tato příloha obsahuje zip archiv s projektem z Visual Studio 2022 jménem TeacherScheduleApp